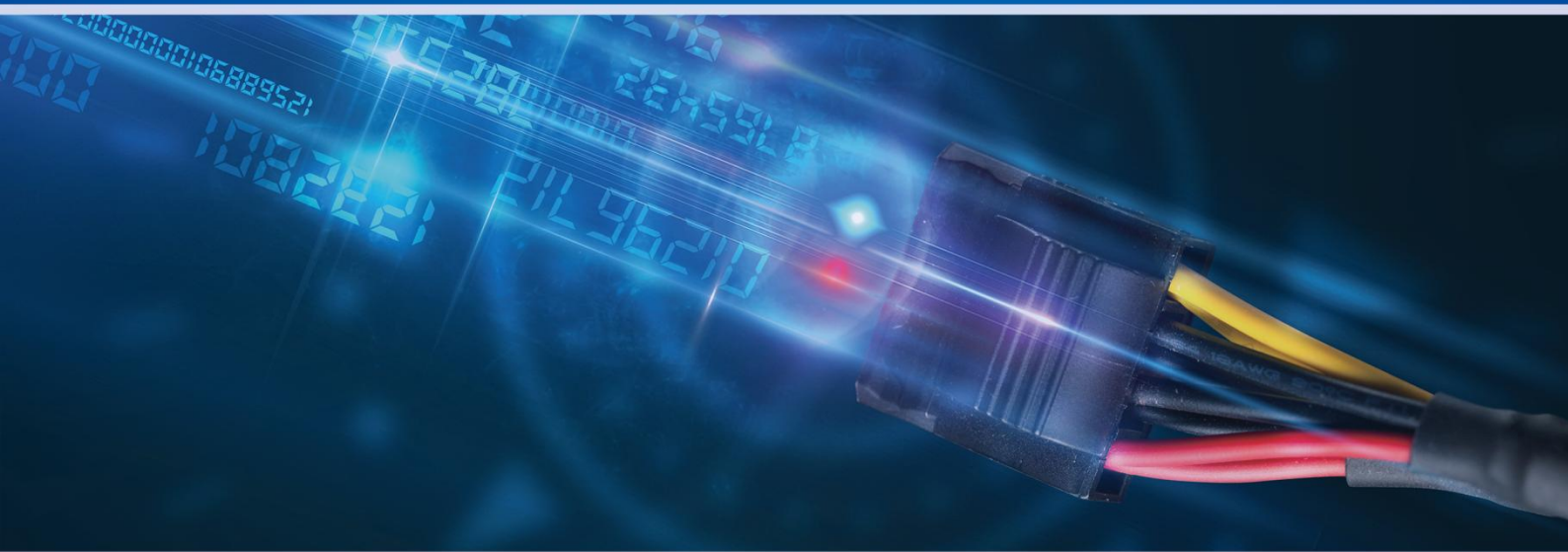




# 华为S系列交换机

## 路由策略专题



初识路由策略



地址前缀列表



Route-Policy详解



filter-policy原理与应用



BGP路由策略

S系列交换机资料组  
交换机与企业网关资料部

## 目录

|       |                                      |    |
|-------|--------------------------------------|----|
| 1     | 初识路由策略.....                          | 1  |
| 1.1   | 路由策略概述.....                          | 1  |
| 1.1.1 | 什么是路由策略?.....                        | 1  |
| 1.1.2 | 路由策略各工具之间的调用关系.....                  | 1  |
| 1.1.3 | 路由策略有什么用?.....                       | 2  |
| 1.2   | 路由策略和策略路由.....                       | 4  |
| 1.2.1 | 路由策略和策略路由的区别.....                    | 4  |
| 1.2.2 | 路由策略和策略路由对比分析.....                   | 4  |
| 1.3   | 路由策略牛刀小试.....                        | 5  |
| 1.3.1 | 通过ACL+route-policy实现路由过滤.....        | 6  |
| 1.3.2 | 通过ip-prefix+filter-policy实现路由过滤..... | 7  |
| 2     | Route-Policy详解.....                  | 9  |
| 2.1   | Route-Policy的组成.....                 | 9  |
| 2.2   | 路由策略匹配结果.....                        | 10 |
| 2.3   | 路由策略使用案例.....                        | 12 |
| 3     | 地址前缀列表.....                          | 16 |
| 3.1   | 地址前缀列表与ACL的区别.....                   | 16 |
| 3.1.1 | 示例1-通过ACL对引入的路由进行过滤.....             | 16 |
| 3.1.2 | 示例2-通过地址前缀列表对引入的路由进行过滤.....          | 18 |
| 3.2   | 地址前缀列表原理及应用.....                     | 20 |
| 3.2.1 | 地址前缀列表的过滤规则.....                     | 20 |
| 3.2.2 | 地址前缀列表中掩码的匹配.....                    | 21 |
| 3.2.3 | 地址前缀列表匹配示例.....                      | 22 |
| 4     | filter-policy原理与应用.....              | 25 |
| 4.1   | filter-policy介绍.....                 | 25 |
| 4.2   | filter-policy过滤路由信息的规则.....          | 25 |



|       |                                        |    |
|-------|----------------------------------------|----|
| 4.3   | 使用filter-policy过滤RIP路由示例 .....         | 27 |
| 4.3.1 | 通过filter-policy对RIP发布的路由做过滤.....       | 27 |
| 4.3.2 | 通过filter-policy对RIP接收的路由做过滤.....       | 29 |
| 4.4   | 使用filter-policy过滤OSPF路由示例 .....        | 31 |
| 4.4.1 | 通过filter-policy对OSPF接收的路由过滤（区域内） ..... | 31 |
| 4.4.2 | 通过filter-policy对OSPF接收的路由过滤（区域间） ..... | 34 |
| 5     | BGP路由策略(上) .....                       | 36 |
| 5.1   | ip-prefix在BGP中的应用 .....                | 36 |
| 5.2   | filter-policy在BGP中的应用 .....            | 37 |
| 5.3   | route-policy在BGP中的应用 .....             | 39 |
| 6     | BGP路由策略(下) .....                       | 43 |
| 6.1   | AS_Path_Filter在BGP中的应用 .....           | 43 |
| 6.1.1 | AS_Path_Filter及正则表达式 .....             | 43 |
| 6.1.2 | AS_Path_Filter的应用方式 .....              | 45 |
| 6.1.3 | AS_Path_Filter的使用举例 .....              | 46 |
| 6.2   | Community属性在BGP中的应用 .....              | 50 |
| 6.2.1 | Community属性介绍 .....                    | 50 |
| 6.2.2 | 设置路由的Community属性 .....                 | 51 |
| 6.2.3 | 使用Community-filter匹配BGP路由 .....        | 54 |

## 1 初识路由策略

对于IP网络工程师来说，路由策略的部署随处可见，无论在运营商IP网络还是在企业网中，路由策略的应用都是非常普遍的。同时，在网络规划中，路由策略的规划也是一个核心的内容。为了方便大家更好的掌握和应用路由策略，我们推出了路由策略这个专题，希望这个专题能够抛砖引玉引导各位一起讨论、共同学习。

### 1.1 路由策略概述

#### 1.1.1 什么是路由策略？

我们讨论某个东西一般都回避不了这样一个问题：“XXX是什么？”这里我们也尝试对路由策略下一个定义，来回答：“路由策略是什么？”这个问题。

很多人会把路由策略等同于route-policy，也有人可能会说filter-policy也属于路由策略的范畴，其实这些理解都有点不太准确。实际上，路由策略不是一个特定的技术，也不是一个特定的特性。

路由策略是通过一系列工具或方法对路由进行各种控制的“策略”。这种策略能够影响到路由产生、发布、选择等，进而影响报文的转发路径。这些工具包括ACL、route-policy、ip-prefix、filter-policy等，这些方法包括对路由进行过滤，设置路由的属性等。

#### 1.1.2 路由策略各工具之间的调用关系

当讨论到路由策略的时候，我们经常会碰到很多种工具，比如ACL、route-policy、ip-prefix、filter-policy等等，不一一列举了。很多人都会被他们之间的调用关系搞昏了头，总感觉他们之间有说清道不明的关系。这里我们通过一张图来给大家介绍他们之间的关系。

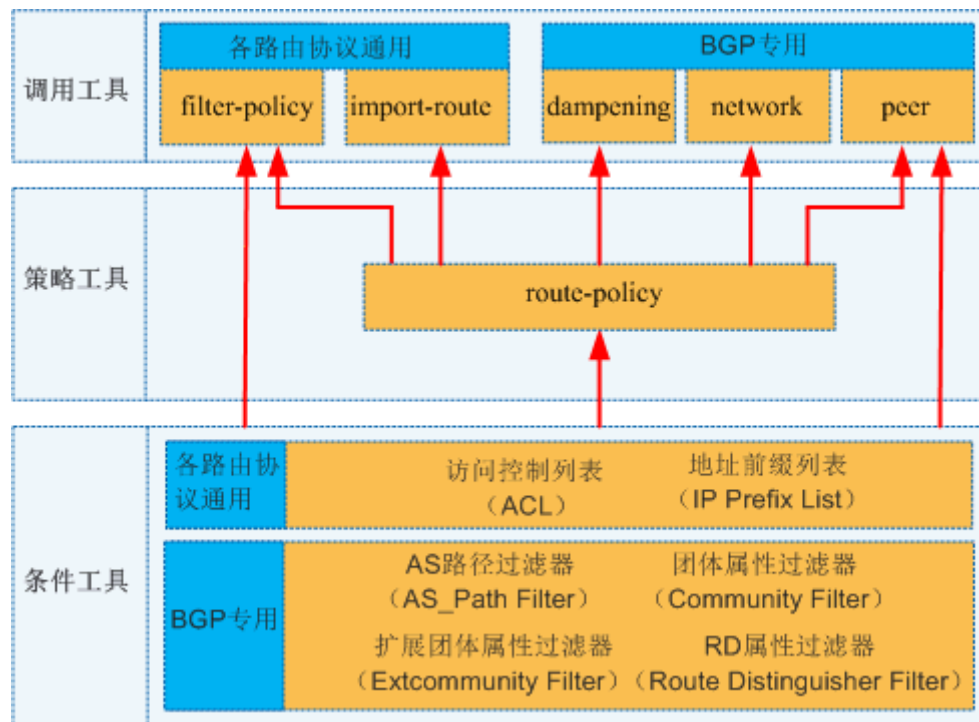


图1 路由策略各工具之间的调用关系

如图1所示，我们把所有的工具划分成三类：

- 条件工具：用于把需要的路由“抓取”出来。
- 策略工具：用于把“抓取”出来的路由执行某个动作，比如允许、拒绝、修改属性值等。
- 调用工具：用于将路由策略应用到某个具体的路由协议里面，使其生效。

调用工具中的filter-policy和peer又自带策略工具的功能，因此这两个东西又可以直接调用条件工具。其他的调用工具都必须通过route-policy来间接的调用条件工具。

需要注意peer不能调用ACL，可以调用其他的所有条件工具。

### 1.1.3 路由策略有什么用？

在IP网络中，路由策略的用途主要包括两个方面：1) 对路由信息进行过滤。2) 修改路由的属性。详细请见表1：

| 作用        | 执行过程                                                                       | 结果       |
|-----------|----------------------------------------------------------------------------|----------|
| 对路由信息进行过滤 | 如果某条路由符合XX条件，那么就接收这条路由<br>如果某条路由符合XX条件，那么就发布这条路由<br>如果某条路由符合XX条件，那么就引入这条路由 | 要不要这条路由？ |

|         |                                  |                |
|---------|----------------------------------|----------------|
| 修改路由的属性 | 如果某条路由符合XX条件，那么将这条路由的某个属性值修改为XXX | 这条路由的某个属性值为XXX |
|---------|----------------------------------|----------------|

表1 路由策略的作用

如果各位觉得这样介绍路由策略的作用还是有点抽象的话，没关系，下面我们再来个实际的例子来介绍一下你就明白了。

### 通过路由策略对路由信息进行过滤

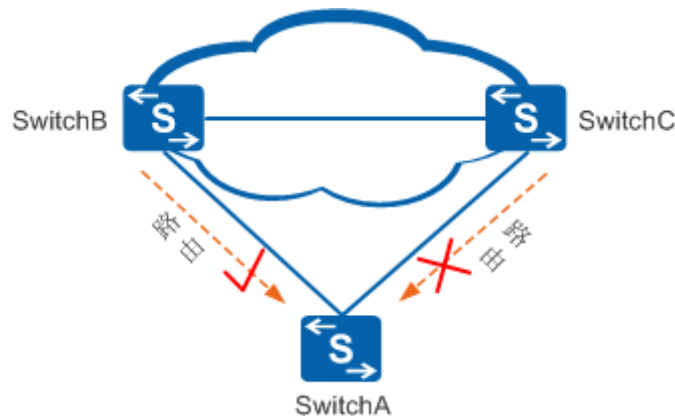


图1 通过路由策略对路由信息进行过滤

如图1所示，SwitchA属于双上行的组网结构，SwitchA会从SwitchB和SwitchC那里分别接收到路由。如果SwitchA仅希望接收来自SwitchB的路由，而不希望接收来自SwitchC的路由，此时应该怎么办呢？这种情况下就可以考虑在SwitchA上配置路由策略，允许来自SwitchB的路由，拒绝来自SwitchC的路由。

### 通过路由策略修改路由的属性

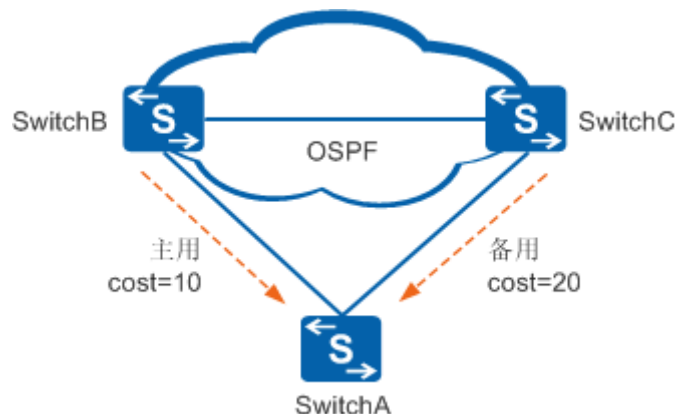


图2 通过路由策略修改路由的属性

如图2所示，SwitchA也是双上行的网络结构，但是，由于SwitchB这边的链路稳定性更好一点，带宽更大一点，因此用户想用SwitchB这边的链路作为主用链路，SwitchC这边的链路作

为备用链路，当主用链路故障的时候流量自动切换至备用链路。这种场景下，可以使用路由策略，将来自SwitchB这边的路由开销值调小，将来自SwitchC这边的路由开销值调大，这样流量就会自动选取SwitchB这边的链路作为主用链路，SwitchC这边的链路作为备用链路，实现路由的主备份。

## 1.2 路由策略和策略路由

### 1.2.1 路由策略和策略路由的区别

我在第一次接触路由策略和策略路由的时候也是抓耳挠腮，分不清楚，老觉得为什么协议的开发者给他们起这么容易混淆的名字，改一个名字就不容易混淆了嘛！但是既然名字叫了这么多年了，各位虽然分不清楚，但已经耳熟了。虽然策略路由这个特性不作为本专题的讨论范畴，我们在这里也把这对孪生兄弟做一个对比分析，让大家不再混淆。

#### 路由策略

路由策略的操作对象是路由信息。路由策略主要实现了路由过滤和路由属性设置等功能，它通过改变路由属性（包括可达性）来改变网络流量所经过的路径。

#### 策略路由

策略路由的操作对象是数据包，在路由表已经产生的情况下，不按照路由表进行转发，而是根据需要，依照某种策略改变数据包转发路径。

所以这样可以看出，策略路由是在路由表之前起作用，如果报文匹配了策略路由，那么这个报文就不会再去查路由表了，而是直接按照策略路由的“指引”进行转发。所以策略路由是一个不太按照套路出牌的“家伙”，也正因为这样，策略路由的应用会更加灵活一点。

### 1.2.2 路由策略和策略路由对比分析

为了更加具体的对比路由策略和策略路由，我们通过表2对两者进行一个全方位的对比。

| 不同点  | 路由策略                     | 策略路由                    |
|------|--------------------------|-------------------------|
| 作用对象 | ● 路由信息                   | ● 数据包                   |
| 实现主体 | ● 控制平面<br>实现路由的过滤和属性值的修改 | ● 转发平面<br>保证数据包按指定的路径转发 |



|          |                                                                                                                                                           |                                                                       |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 是否改变转发流程 | <ul style="list-style-type: none"> <li>● 不改变转发流程</li> </ul>                                                                                               | <ul style="list-style-type: none"> <li>● 改变了数据包转发流程</li> </ul>        |
| 过滤机制     | <ul style="list-style-type: none"> <li>● 基于ACL过滤</li> <li>● 基于地址前缀列表</li> <li>● 基于路由属性过滤</li> <li>● 基于路由类型过滤</li> <li>.....</li> </ul>                    | <ul style="list-style-type: none"> <li>● 基于ACL、流分类、流行为、流策略</li> </ul> |
| 应用主体     | <ul style="list-style-type: none"> <li>● 静态路由</li> <li>● 直连路由</li> <li>● RIP/RIPng</li> <li>● OSPF/OSPFv3</li> <li>● ISIS</li> <li>● BGP/BGP4+</li> </ul> | <ul style="list-style-type: none"> <li>● 全局、vlan、接口下应用</li> </ul>     |

表2 路由策略和策略路由对比分析

### 1.3 路由策略牛刀小试

上面在宏观上介绍了关于路由策略的一些基础知识，各位是不是还是觉得有点不过瘾？是不是还感觉不到路由策略究竟有什么洪荒之力？没关系，接下来我们来看一个通过路由策略实现路由过滤的举例，算作牛刀小试。这个举例中会涉及ACL、ip-prefix、route-policy、filter-policy等概念，我们会在后面几期的专题中详细展开介绍，各位就先了解一下路由策略究竟能干什么就行，先不要研究太深，以免走火入魔！

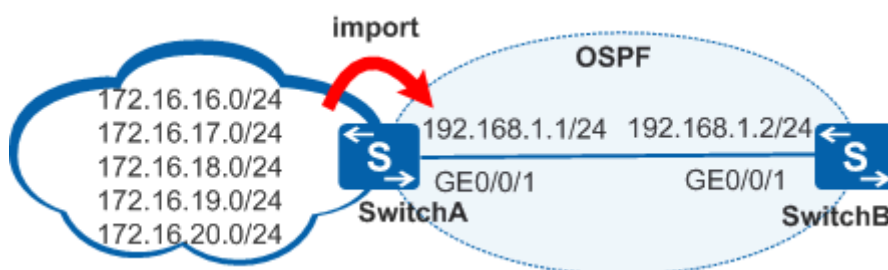


图3 通过路由策略实现路由过滤示例

如图3所示，运行OSPF协议的网络中，SwitchA从Internet网络接收路由，并为OSPF网络提供了Internet路由，现在用户希望OSPF网络仅接收172.16.16.0/24、172.16.17.0/24和



172.16.18.0/24这三条外部路由，其他的外部路由都不接收。

上述用户需求可以通过多种方式去实现，接下来我们给出两个比较常见的实现方式供各位参考。

下面的实验中我们通过在SwitchA中配置黑洞路由做为测试路由，在OSPF中引入静态路由来模拟从Internet网络接收路由。SwitchA上的关键配置如下：

```
#
ospf 1
import-route static
area 0.0.0.0
network 192.168.1.0 0.0.0.255
#
ip route-static 172.16.16.0 255.255.255.0 NULL0
ip route-static 172.16.17.0 255.255.255.0 NULL0
ip route-static 172.16.18.0 255.255.255.0 NULL0
ip route-static 172.16.19.0 255.255.255.0 NULL0
ip route-static 172.16.20.0 255.255.255.0 NULL0
#
```

### 1.3.1 通过 ACL+route-policy 实现路由过滤

1、定义一个ACL 2000，用于匹配需要放行的路由。

```
[SwitchA] acl 2000
[SwitchA-acl-basic-2000] rule 5 permit source 172.16.16.0 0
[SwitchA-acl-basic-2000] rule 10 permit source 172.16.17.0 0
[SwitchA-acl-basic-2000] rule 15 permit source 172.16.18.0 0
[SwitchA-acl-basic-2000] quit
```

2、创建一个route-policy，名字叫RP，同时配置一个编号为10的节点，调用ACL2000。

```
[SwitchA] route-policy RP permit node 10
[SwitchA -route-policy] if-match acl 2000
```

3、在OSPF引入静态路由的时候调用这个route-policy

```
[SwitchA] ospf 1
[SwitchA-ospf-1] import-route static route-policy RP
[SwitchA-ospf-1] quit
```

由于route-policy默认隐含deny节点，因此172.16.19.0及172.16.20.0路由由于没有满足if-match语句，从而不被引入到OSPF中。

配置完上述路由策略以后SwitchB的路由表如下：

```
[SwitchB]display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----  
Routing Tables: Public
```

```
Destinations : 7      Routes : 7
```

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop     | Interface   |
|------------------|--------|-----|------|-------|-------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1   | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1   | InLoopBack0 |
| 172.16.16.0/24   | O_ASE  | 150 | 1    | D     | 192.168.1.1 | Vlanif10    |
| 172.16.17.0/24   | O_ASE  | 150 | 1    | D     | 192.168.1.1 | Vlanif10    |
| 172.16.18.0/24   | O_ASE  | 150 | 1    | D     | 192.168.1.1 | Vlanif10    |
| 192.168.1.0/24   | Direct | 0   | 0    | D     | 192.168.1.2 | Vlanif10    |
| 192.168.1.2/32   | Direct | 0   | 0    | D     | 127.0.0.1   | Vlanif10    |

可以看到在SwitchA上配置完路由策略以后，SwitchB的IP路由表里面只有172.16.16.0/24、172.16.17.0/24和172.16.18.0/24这三条外部路由，其他的外部路由都没有了。

### 1.3.2 通过 ip-prefix+filter-policy 实现路由过滤

1、定义一个地址前缀列表，用于匹配需要放行的路由。

```
[SwitchA] ip ip-prefix huawei index 10 permit 172.16.16.0 24  
[SwitchA] ip ip-prefix huawei index 20 permit 172.16.17.0 24  
[SwitchA] ip ip-prefix huawei index 30 permit 172.16.18.0 24
```

2、在SwitchA的OSPF视图中，通过filter-policy对发布的路由进行过滤。

```
[SwitchA] ospf 1  
[SwitchA-ospf-1] filter-policy ip-prefix huawei export  
[SwitchA-ospf-1] quit
```

由于ip-prefix默认隐含deny节点，因此172.16.19.0及172.16.20.0路由由于不在ip-prefix允许的范围内，所以在SwitchA向SwitchB发布路由的时候，仅发布在ip-prefix允许的范围内路由，其他的所有路由都不发布。

配置完上述配置以后SwitchB的路由表如下：

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
    Destinations : 7          Routes : 7

Destination/Mask    Proto   Pre  Cost           Flags NextHop         Interface
-----
127.0.0.0/8        Direct  0    0              D  127.0.0.1          InLoopBack0
127.0.0.1/32        Direct  0    0              D  127.0.0.1          InLoopBack0
172.16.16.0/24      O_ASE   150  1              D  192.168.1.1        Vlanif10
172.16.17.0/24      O_ASE   150  1              D  192.168.1.1        Vlanif10
172.16.18.0/24      O_ASE   150  1              D  192.168.1.1        Vlanif10
192.168.1.0/24      Direct  0    0              D  192.168.1.2        Vlanif10
192.168.1.2/32      Direct  0    0              D  127.0.0.1          Vlanif10
```

可以看到在SwitchA上配置完路由filter-policy以后，SwitchB的IP路由表里面只有172.16.16.0/24、172.16.17.0/24和172.16.18.0/24这三条外部路由，其他的外部路由都没有了。从实验结果来看，上述两种方法使用的工具和方法不同，但是结果是一样的。相信各位是不是已经看到路由策略确实不能直接等同于route-policy了吧？实际上，路由策略是一系列对路由进行控制的手段，路由策略的使用过程中可能是ACL、route-policy、ip-prefix、filter-policy等多个工具的不同组合，上述举例仅仅列举了其中两种比较常见的组合而已。在后面几期的专题中，我们会深入分析各种工具的使用方法，相信各位全部掌握这些工具之后就会对路由策略的使用做到游刃有余、随心所欲了。

## 2 Route-Policy 详解

在上一期的路由策略专题中，我们曾经提到，很多人会把路由策略等同于Route-Policy，虽然这不太准确，但是能够体现出Route-Policy的重要性和普遍性。Route-Policy是一种比较复杂的过滤器，它不仅可以匹配路由信息的某些属性，还可以在条件满足时改变路由信息的属性。这一期我们就详细介绍一下Route-Policy，包括Route-Policy的组成、匹配规则和使用举例等内容。

### 2.1 Route-Policy 的组成

如图1所示，Route-Policy由节点号、匹配模式、if-match子句（条件语句）和apply子句（执行语句）这四个部分组成。

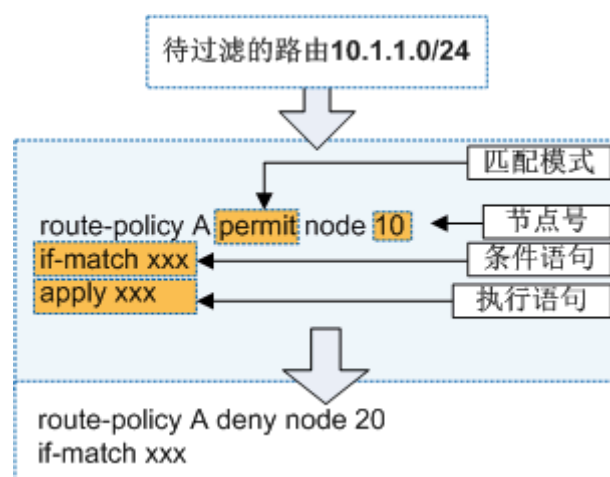


图1 Route-Policy的组成

#### ● 节点号

一个Route-Policy可以由多个节点（node）构成。路由匹配Route-Policy时遵循以下两个规则：

- 1) 顺序匹配：在匹配过程中，系统按节点号从小到大的顺序依次检查各个表项，因此在指定节点号时，要注意符合期望的匹配顺序。
- 2) 唯一匹配：Route-Policy各节点号之间是“或”的关系，只要通过一个节点的匹配，就认为通过该过滤器，不再进行其它节点的匹配。

#### ● 匹配模式

节点的匹配模式有两种：permit和deny。

- 1) permit指定节点的匹配模式为允许。当路由项通过该节点的过滤后，将执行该节点的apply子句，不进入下一个节点；如果路由项没有通过该节点过滤，将进入下一个节点继续匹配。
- 2) deny指定节点的匹配模式为拒绝。这时apply子句不会被执行。当路由项满足该节点的所有if-match子句时，将被拒绝通过该节点，不进入下一个节点；如果路由项不满足该节点的if-match子句，将进入下一个节点继续匹配。

**注意事项：**

通常在多个deny节点后设置一个不含if-match子句和apply子句的permit模式的Route-Policy，用于允许其它所有的路由通过。

**● if-match子句（条件语句）**

if-match子句用来定义一些匹配条件。Route-Policy的每一个节点可以含有多个if-match子句，也可以不含if-match子句。如果某个permit节点没有配置任何if-match子句，则该节点匹配所有的路由。

**● apply子句（执行语句）**

apply子句用来指定动作。路由通过Route-Policy过滤时，系统按照apply子句指定的动作对路由信息的一些属性进行设置。Route-Policy的每一个节点可以含有多个apply子句，也可以不含apply子句。如果只需要过滤路由，不需要设置路由的属性，则不使用apply子句。

## 2.2 路由策略匹配结果

相信大家在使用或者学习Route-Policy的时候都重点关注这样一个问题：对于一条路由，在使用Route-Policy以后，最终结果是允许还是拒绝这条路由呢？这个最终的结果对于业务的影响是非常大的，可能会直接影响某种业务的通与不通。这就涉及到Route-Policy匹配规则的问题了，这里我们详细讨论一下。

**Route-Policy每个node节点的过滤结果要综合以下两点：**

- 1、Route-Policy的node节点的匹配模式（permit或deny）。
- 2、if-match子句（如引用的地址前缀列表或者访问控制列表）中包含的匹配条件（permit或deny）。

对于每一个node节点，以上两点的排列组合会出现表1所示的4种情况。

| Rule | Mode | 匹配结果 |
|------|------|------|
|------|------|------|

|                                                                                                            |        |                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| permit                                                                                                     | permit | <ul style="list-style-type: none"> <li>匹配该节点if-match子句的路由在本节点允许通过Route-Policy，匹配结束。</li> <li>不匹配if-match子句的路由进行Route-Policy下一个节点的匹配。</li> </ul>                      |
| permit                                                                                                     | deny   | <ul style="list-style-type: none"> <li>匹配该节点if-match子句的路由在本节点不允许通过Route-Policy，匹配结束。</li> <li>不匹配if-match子句的路由进行Route-Policy下一个节点的匹配。</li> </ul>                     |
| deny                                                                                                       | permit | <ul style="list-style-type: none"> <li>匹配该节点if-match子句的路由在本节点不允许通过Route-Policy，继续进行Route-Policy下一个节点的匹配。</li> <li>不匹配if-match子句的路由进行Route-Policy下一个节点的匹配。</li> </ul> |
| deny                                                                                                       | deny   | <ul style="list-style-type: none"> <li>匹配该节点if-match子句的路由在本节点不允许通过Route-Policy，继续进行Route-Policy下一个节点的匹配。</li> <li>不匹配if-match子句的路由进行Route-Policy下一个节点的匹配。</li> </ul> |
| <p>注1: Rule表示if-match子句中包含的匹配模式是permit还是deny。</p> <p>注2: Mode表示Route-Policy中node节点对应的匹配模式permit还是deny。</p> |        |                                                                                                                                                                      |

表1 Route-Policy的匹配规则

上述四种组合情况中，前两种比较好理解，也比较常用。后两种相对难理解一点，这里我们以第三种情况为例，举例说明一下：

假设if-match子句中包含的匹配条件是deny，node节点对应的匹配条件permit，配置如下：

```
#
acl number 2001
 rule 5 deny source 172.16.16.0 0 //拒绝 172.16.16.0
#
acl number 2002
 rule 5 permit source 172.16.16.0 0 //允许 172.16.16.0
#
route-policy RP permit node 10 //在这个节点，172.16.16.0 这条路由被拒绝，继续往下
 if-match acl 2001
#
route-policy RP permit node 20 //在这个节点，172.16.16.0 这条路由被允许
 if-match acl 2002
#
```

这种情况下，有一个关键点就是在node 10,172.16.16.0这条路由被拒绝，同时会继续往下匹

配，或许下一个节点就允许通过了呢？果然，继续往下走，到node 20这个节点的时候172.16.16.0又被允许了，所以Route-Policy的最终匹配结果是允许172.16.16.0这条路由。

### 注意事项：

华为S交换机默认所有未匹配的路由将被拒绝通过Route-Policy。如果Route-Policy中定义了一个以上的节点，应保证各节点中至少有一个节点的匹配模式是permit。因为Route-Policy用于路由信息过滤时：

- 如果某路由信息没有通过任一节点，则认为该路由信息没有通过该Route-Policy。
- 如果Route-Policy的所有节点都是deny模式，则没有路由信息能通过该Route-Policy。

## 2.3 路由策略使用案例

通过上面两个小节，我们介绍完了Route-Policy的组成和匹配规则。这个小节中，我们来看一个Route-Policy的使用实例。

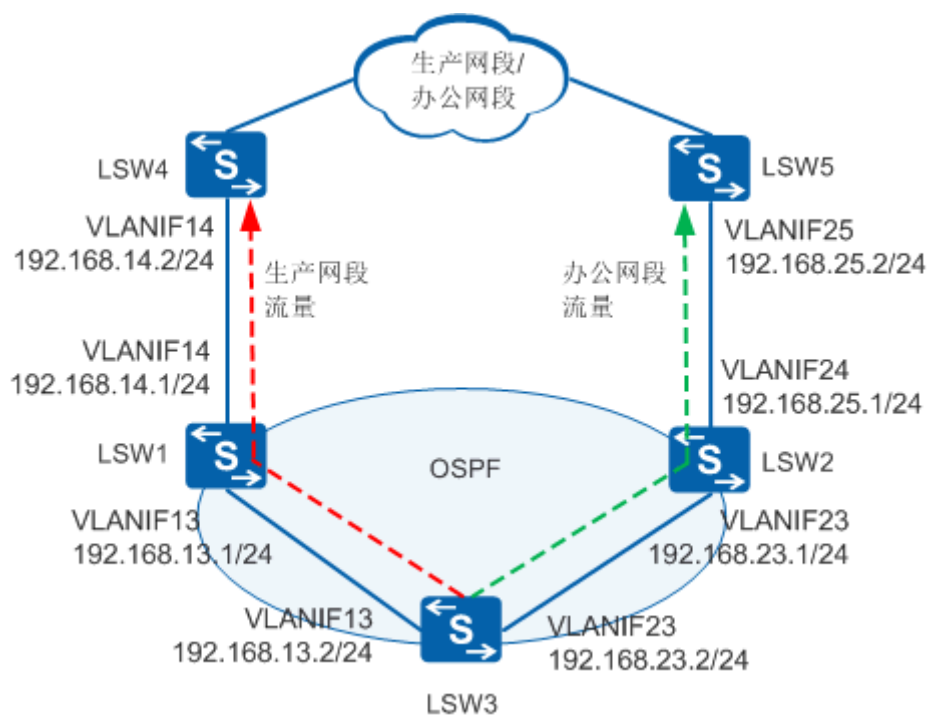


图2 使用Route-Policy实现数据分流示例

### 用户需求

如图2所示，某园区网络主要划分为生产网段和办公网段。LSW3下挂的终端访问下面的网段的时候流量模型如下：

10.10.1.0/24-----生产网段，优先走LSW1出去，LSW2作为备份链路。



10.10.2.0/24-----办公网段，优先走LSW2出去，LSW1作为备份链路。

10.10.3.0/24-----其他网段，随便走那边都行，负载分担即可。

这种流量模型，可以保证生产网络与办公网络的流量分离，便于维护和故障定位。同时，这种流量模型有利于流量均衡的分配到两条链路上，同时互相作为备份链路，有利于网络的稳定性。

### 配置过程

- 1、LSW1、LSW2、LSW3三个设备之间建立OSPF邻居关系。
- 2、LSW1和LSW2上配置到达上述网段的静态路由，并引入OSPF，从而通告给LSW3。
- 3、LSW1和LSW2上配置路由策略，调整流量模型满足用户规划的需求。

这里仅给出涉及路由策略的关键配置：

#### LSW1关键配置：

```
#
acl number 2000
  rule 5 permit source 10.10.1.0 0 //用于匹配生产网段路由
#
acl number 2001
  rule 5 permit source 10.10.2.0 0 //用于匹配办公网段路由
#
route-policy RP permit node 10
  if-match acl 2000
  apply cost 10 //设置生产网段路由的 cost 值为 10
#
route-policy RP permit node 20
  if-match acl 2001
  apply cost 20 //设置办公网段路由的 cost 值为 20
#
route-policy RP permit node 30 //剩余网段的路由允许进来，不做任何处理
#
ip route-static 10.10.1.0 255.255.255.0 192.168.14.2
ip route-static 10.10.2.0 255.255.255.0 192.168.14.2
ip route-static 10.10.3.0 255.255.255.0 192.168.14.2
#
```

#### LSW2关键配置：

```
#
acl number 2000
  rule 5 permit source 10.10.1.0 0 //用于匹配生产网段路由
```

```
#
acl number 2001
 rule 5 permit source 10.10.2.0 0 //用于匹配办公网段路由
#
route-policy RP permit node 10
 if-match acl 2000
 apply cost 20 //设置生产网段路由的 cost 值为 20
#
route-policy RP permit node 20
 if-match acl 2001
 apply cost 10 //设置办公网段路由的 cost 值为 10
#
route-policy RP permit node 30 //剩余网段的路由允许进来，不做任何处理
#
ip route-static 10.10.1.0 255.255.255.0 192.168.25.2
ip route-static 10.10.2.0 255.255.255.0 192.168.25.2
ip route-static 10.10.3.0 255.255.255.0 192.168.25.2
#
```

## 结果验证

完成上述配置以后，可以在LSW3上查看IP路由表，确认流量模型是否正确。

```
<LSW3> display ip routing-table
```

Route Flags: R - relay, D - download to fib

-----  
-  
Routing Tables: Public

Destinations : 9 Routes : 10

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 10.10.1.0/24     | O_ASE  | 150 | 10   | D     | 192.168.13.1 | Vlanif13    |
| 10.10.2.0/24     | O_ASE  | 150 | 10   | D     | 192.168.23.1 | Vlanif23    |
| 10.10.3.0/24     | O_ASE  | 150 | 1    | D     | 192.168.23.1 | Vlanif23    |
|                  | O_ASE  | 150 | 1    | D     | 192.168.13.1 | Vlanif13    |
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.13.0/24  | Direct | 0   | 0    | D     | 192.168.13.2 | Vlanif13    |
| 192.168.13.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif13    |
| 192.168.23.0/24  | Direct | 0   | 0    | D     | 192.168.23.2 | Vlanif23    |
| 192.168.23.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif23    |

从LSW3的路由表中可以看到，到达生产网段10.10.1.0/24的流量优先走LSW1，到达办公网段10.10.2.0/24的流量优先走LSW2，到达其他网段的流量在LSW1和LSW2两条链路上进行负



载分担。流量模型符合预期。

通过本期专题，我们把Route-Policy的组成结构、匹配规则基本上就讲清楚了，也通过一个实例让大家了解了Route-Policy的使用场景和配置方法。本期专题中，我们主要使用了ACL来“抓取”需要的路由，实际上地址前缀列表（ip ip-prefix）在“抓取”路由方面会更精确一点，这个我们将会在下期的路由策略专题中详细介绍。

## 3 地址前缀列表

想对接收的路由/发布的路由/引入的路由进行过滤，或者设置相关的路由属性？第一步当然要先筛选出想要的路由，通过ACL或者地址前缀列表（ip ip-prefix）都可以实现。在上一期 route-policy 的介绍中我们就是采用ACL来筛选路由的，ACL大家平时接触的也比较多，但地址前缀列表是什么、怎么用？地址前缀列表和ACL有什么区别？这一期的路由策略专题我们将会详细介绍这些内容。

### 3.1 地址前缀列表与 ACL 的区别

让我们先来看两个具体的例子。

#### 3.1.1 示例 1-通过 ACL 对引入的路由进行过滤

如图1所示，通过ACL实现将RIP中的2条路由引入到OSPF中，并设置路由的开销值。

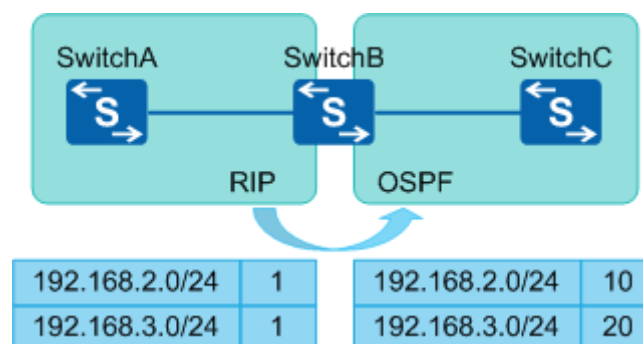


图1 通过ACL对引入的路由进行过滤

查看SwitchB上的路由表，有2条RIP路由192.168.2.0/24和192.168.3.0/24，想将这2条路由重发布到OSPF中，并将192.168.2.0/24这条路由的开销值设置为10，192.168.3.0/24这条路由的开销值设置为20。

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
      Destinations : 8          Routes : 8

Destination/Mask    Proto   Pre  Cost           Flags NextHop         Interface
```

```

10.1.1.0/24 Direct 0 0 D 10.1.1.1 Vlanif20
10.1.1.1/32 Direct 0 0 D 127.0.0.1 Vlanif20
127.0.0.0/8 Direct 0 0 D 127.0.0.1 InLoopBack0
127.0.0.1/32 Direct 0 0 D 127.0.0.1 InLoopBack0
192.168.1.0/24 Direct 0 0 D 192.168.1.2 Vlanif10
192.168.1.2/32 Direct 0 0 D 127.0.0.1 Vlanif10
192.168.2.0/24 RIP 100 1 D 192.168.1.1 Vlanif10
192.168.3.0/24 RIP 100 1 D 192.168.1.1 Vlanif10

```

## 1. 配置ACL，过滤出想要的路由

# 配置基本ACL 2001，匹配路由网络号192.168.2.0。

```

[SwitchB] acl 2001
[SwitchB-acl-basic-2001] rule permit source 192.168.2.0 0
[SwitchB-acl-basic-2001] quit

```

# 配置基本ACL 2002，匹配路由网络号192.168.3.0。

```

[SwitchB] acl 2002
[SwitchB-acl-basic-2002] rule permit source 192.168.3.0 0
[SwitchB-acl-basic-2002] quit

```

## 2. 配置route-policy，并对引入的路由应用route-policy

# 配置route-policy RP的节点10，如果匹配基本ACL 2001，则设置路由开销值为10。

```

[SwitchB] route-policy RP permit node 10
[SwitchB-route-policy] if-match acl 2001
[SwitchB-route-policy] apply cost 10
[SwitchB-route-policy] quit

```

# 配置route-policy RP的节点20，如果匹配基本ACL 2002，则设置路由开销值为20。

```

[SwitchB] route-policy RP permit node 20
[SwitchB-route-policy] if-match acl 2002
[SwitchB-route-policy] apply cost 20
[SwitchB-route-policy] quit

```

# 配置在OSPF路由中引入通过route-policy RP过滤的RIP路由。

```

[SwitchB] OSPF
[SwitchB-ospf-1] import-route rip 1 route-policy RP
[SwitchB-ospf-1] quit

```

配置完成后，在SwitchC上查看路由表，发现已经成功引入了2条RIP路由，并且路由开销值已进行了相应的设置。

```

<SwitchC> display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
      Destinations : 6          Routes : 6

```

| Destination/Mask      | Proto        | Pre        | Cost      | Flags    | NextHop         | Interface       |
|-----------------------|--------------|------------|-----------|----------|-----------------|-----------------|
| 10.1.1.0/24           | Direct       | 0          | 0         | D        | 10.1.1.2        | Vlanif20        |
| 10.1.1.2/32           | Direct       | 0          | 0         | D        | 127.0.0.1       | Vlanif20        |
| 127.0.0.0/8           | Direct       | 0          | 0         | D        | 127.0.0.1       | InLoopBack0     |
| 127.0.0.1/32          | Direct       | 0          | 0         | D        | 127.0.0.1       | InLoopBack0     |
| <b>192.168.2.0/24</b> | <b>O_ASE</b> | <b>150</b> | <b>10</b> | <b>D</b> | <b>10.1.1.1</b> | <b>Vlanif20</b> |
| <b>192.168.3.0/24</b> | <b>O_ASE</b> | <b>150</b> | <b>20</b> | <b>D</b> | <b>10.1.1.1</b> | <b>Vlanif20</b> |

### 3.1.2 示例 2-通过地址前缀列表对引入的路由进行过滤

如图2所示，SwitchB上有2条静态路由，如果只想将192.168.0.0/16这条路由引入OSPF中，该怎么配置呢？

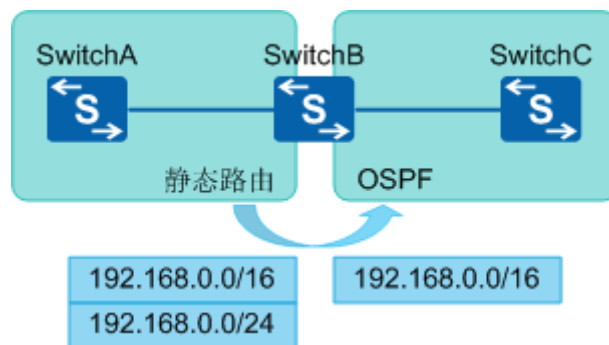


图2 通过地址前缀列表对引入的路由进行过滤

查看SwitchB上的路由表，有2条静态路由192.168.0.0/16和192.168.0.0/24，只想将192.168.0.0/16这条路由重发布到OSPF中。

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib
-----
```

| Destination/Mask      | Proto         | Pre       | Cost     | Flags    | NextHop        | Interface    |
|-----------------------|---------------|-----------|----------|----------|----------------|--------------|
| 10.10.12.0/24         | Direct        | 0         | 0        | D        | 10.10.12.1     | Vlanif10     |
| 10.10.12.1/32         | Direct        | 0         | 0        | D        | 127.0.0.1      | Vlanif10     |
| 127.0.0.0/8           | Direct        | 0         | 0        | D        | 127.0.0.1      | InLoopBack0  |
| 127.0.0.1/32          | Direct        | 0         | 0        | D        | 127.0.0.1      | InLoopBack0  |
| <b>192.168.0.0/16</b> | <b>Static</b> | <b>60</b> | <b>0</b> | <b>D</b> | <b>0.0.0.0</b> | <b>NULL0</b> |
| <b>192.168.0.0/24</b> | <b>Static</b> | <b>60</b> | <b>0</b> | <b>D</b> | <b>0.0.0.0</b> | <b>NULL0</b> |

首先我们尝试用ACL来实现。

## 1. 配置基本ACL 2001

```
[SwitchB] acl 2001
[SwitchB-acl-basic-2001] rule permit source 192.168.0.0 0.0.255.255
[SwitchB-acl-basic-2001] quit
```

## 2. 配置route-policy，并对引入的路由应用route-policy

# 配置route-policy RP的节点10，如果匹配基本ACL 2001，则允许通过。其他所有未匹配成功的路由都被拒绝通过。

```
[SwitchB] route-policy RP permit node 10
[SwitchB-route-policy] if-match acl 2001
[SwitchB-route-policy] quit
```

# 配置在OSPF路由中引入通过route-policy RP过滤的静态路由。

```
[SwitchB] OSPF
[SwitchB-ospf-1] import-route static route-policy RP
[SwitchB-ospf-1] quit
```

配置完成后，在SwitchC上查看路由表，发现有2条192.168.0.0网段的路由，2条路由都被引入了。这是由于ACL2001规则rule permit source 192.168.0.0 0.0.255.255中，0.0.255.255实际上是通配符，而不是掩码长度。

所谓通配符，就是指换算成二进制后，“0”表示需要匹配，“1”表示不需要匹配。例如192.168.0.0 0.0.255.255表示匹配网络号192.168.0.0~192.168.255.255，192.168.0.0/16和192.168.0.0/24都能匹配成功ACL2001，因此这2条路由匹配了route-policy RP的节点10，都被引入了。ACL无法实现只匹配192.168.0.0/16或者只匹配192.168.0.0/24，ACL只能匹配网络号，无法匹配掩码。

```
<SwitchC> display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
      Destinations : 6          Routes : 6

Destination/Mask    Proto   Pre  Cost           Flags NextHop         Interface
-----
10.10.12.0/24      Direct   0    0             D   10.10.12.2        Vlanif10
10.10.12.2/32      Direct   0    0             D   127.0.0.1         Vlanif10
127.0.0.0/8        Direct   0    0             D   127.0.0.1         InLoopBack0
127.0.0.1/32       Direct   0    0             D   127.0.0.1         InLoopBack0
192.168.0.0/16     O_ASE    150   1             D   10.10.12.1        Vlanif10
192.168.0.0/24     O_ASE    150   1             D   10.10.12.1        Vlanif10
```

接下来，我们试试用地址前缀列表对引入的路由进行过滤，看是否能实现引入192.168.0.0/16这条路由，过滤掉192.168.0.0/24。

## 1. 配置地址前缀列表，过滤出想要的路由



# 地址前缀列表为huawei，节点号10，允许192.168.0.0/16的路由通过。

```
[SwitchB] ip ip-prefix huawei index 10 permit 192.168.0.0 16
```

2. 配置route-policy，并对引入的路由应用route-policy

# 配置route-policy RP的节点10，如果匹配地址前缀列表huawei，则允许通过；其他所有未匹配上的路由都将默认拒绝。

```
[SwitchB] route-policy RP permit node 10
[SwitchB-route-policy] if-match ip-prefix huawei
[SwitchB-route-policy] quit
```

# 配置在OSPF路由中引入通过route-policy RP过滤的静态路由。

```
[SwitchB] OSPF
[SwitchB-ospf-1] import-route static route-policy RP
[SwitchB-ospf-1] quit
```

配置完成后，在SwitchC上查看路由表，成功实现了只引入192.168.0.0/16这1条路由。

```
<SwitchC> display ip routing-table
```

```
Route Flags: R - relay, D - download to fib
```

```
-----
Routing Tables: Public
```

```
Destinations : 5      Routes : 5
```

| Destination/Mask      | Proto        | Pre        | Cost     | Flags    | NextHop           | Interface       |
|-----------------------|--------------|------------|----------|----------|-------------------|-----------------|
| 10.10.12.0/24         | Direct       | 0          | 0        | D        | 10.10.12.2        | Vlanif10        |
| 10.10.12.2/32         | Direct       | 0          | 0        | D        | 127.0.0.1         | Vlanif10        |
| 127.0.0.0/8           | Direct       | 0          | 0        | D        | 127.0.0.1         | InLoopBack0     |
| 127.0.0.1/32          | Direct       | 0          | 0        | D        | 127.0.0.1         | InLoopBack0     |
| <b>192.168.0.0/16</b> | <b>O_ASE</b> | <b>150</b> | <b>1</b> | <b>D</b> | <b>10.10.12.1</b> | <b>Vlanif10</b> |



总结一下上面的两个示例，ACL和地址前缀列表都可以对路由进行筛选，

ACL匹配路由时只能匹配路由的网络号，但无法匹配掩码，也就是前缀长度；而地址前缀列表比ACL更为灵活，可以匹配路由的网络号及掩码，增强了路由匹配的精确度。

## 3.2 地址前缀列表原理及应用

### 3.2.1 地址前缀列表的过滤规则

一个地址前缀列表中创建多个索引项，每个索引对应一条过滤规则。如图3所示，待过

滤路由按照索引号从小到大的顺序进行匹配：

- ✧ 当匹配上某一索引项时，如果该索引项是permit，则这条路由被允许通过；如果该索引项是deny，则这条路由被拒绝通过。
- ✧ 当遍历了地址前缀列表中的所有索引项，都没有匹配上，那么这条路由就被拒绝通过。

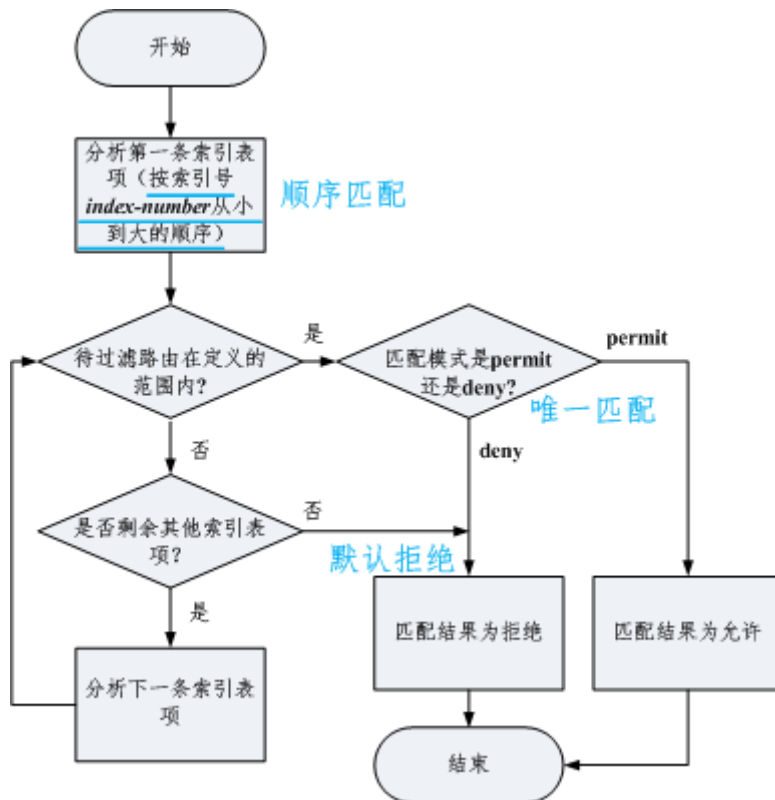


图3 地址前缀列表原理

地址前缀列表过滤路由的原则可以总结为：顺序匹配、唯一匹配、默认拒绝。

**顺序匹配：**按索引号从小到大顺序进行匹配。同一个地址前缀列表中的多条表项设置不同的索引号，可能会有不同的过滤结果，实际配置时需要注意。

**唯一匹配：**待过滤路由只要与一个表项匹配，就不会再去尝试匹配其他表项。

**默认拒绝：**默认所有未与任何一个表项匹配的路由都视为未通过地址前缀列表的过滤。因此在一个地址前缀列表中创建了一个或多个deny模式的表项后，需要创建一个表项来允许所有其他路由通过。

### 3.2.2 地址前缀列表中掩码的匹配

地址前缀列表与ACL相比的一大优势就是可以对路由的掩码进行匹配，在前面的示例中我们

已经用到了精确匹配路由中的掩码。不仅如此，地址前缀列表还可以匹配一个掩码范围。

地址前缀列表通过 **ip ip-prefix** 命令进行配置，常用格式如下：

```
ip ip-prefix ip-prefix-name [ index index-number ] { permit | deny } ipv4-address mask-length  
[ greater-equal greater-equal-value ] [ less-equal less-equal-value ]
```

其中 *ipv4-address mask-length* [ **greater-equal** *greater-equal-value* ] [ **less-equal** *less-equal-value* ] 用于限定过滤路由的网络号及掩码范围，参数含义如表1所示。

| 参数                                              | 含义                                        |
|-------------------------------------------------|-------------------------------------------|
| <i>ipv4-address</i>                             | 用于指定网络号                                   |
| <i>mask-length</i>                              | 用于限定网络号的前多少位需严格匹配                         |
| <b>greater-equal</b> <i>greater-equal-value</i> | 可以理解为掩码 $\geq$ <i>greater-equal-value</i> |
| <b>less-equal</b> <i>less-equal-value</i>       | 可以理解为掩码 $\leq$ <i>less-equal-value</i>    |

表1 地址前缀列表中地址范围的表示

当待过滤的路由已匹配当前表项的网络号时，掩码长度可以进行精确匹配或者在一定掩码长度度范围内匹配。

- 若不配置 **greater-equal** 和 **less-equal**，则进行精确匹配，即只匹配掩码长度为 *mask-length* 的路由。
- 若只配置 **greater-equal**，则匹配的掩码长度范围为 [*greater-equal-value*, 32]。
- 若只配置 **less-equal**，则匹配的掩码长度范围为 [*mask-length*, *less-equal-value*]。
- 若同时配置 **greater-equal** 和 **less-equal**，则匹配的掩码长度范围为 [*greater-equal-value*, *less-equal-value*]。

### 3.2.3 地址前缀列表匹配示例

好啦，学了这么多的理论，接下来让我们实际操练一下试试。假设有这么几条路由 10.1.1.0/24、10.1.1.0/26、10.1.1.1/32、10.2.2.0/24 和 10.1.0.0/16，你们有没有办法用地址前缀列表筛选出想要的路由呢？

1. 只想 permit 某1条路由？例如只 permit 10.1.1.0/24 这1条路由。
2. 只想 permit 网络号相同，掩码不同的某几条路由，其他路由都 deny？例如只 permit 10.1.1.0/24、10.1.1.0/26、10.1.1.1/32 这3条路由。



3. 只想deny某1条路由，其他路由都permit？例如只deny10.1.1.0/24这1条路由。

答案请在下面示例中找~

-----示例1为单节点精确匹配示例-----

✓ 示例1:

```
ip ip-prefix test index 10 permit 10.1.1.0 24
```

匹配结果：只有路由由10.1.1.0/24被permit，其他路由都被deny

说明：只有网络号、掩码完全相同的路由才会匹配成功。

-----示例2-4为指定掩码匹配范围示例-----

✓ 示例2:

```
ip ip-prefix test index 10 permit 10.1.1.0 24 less-equal 32
```

匹配结果：路由10.1.1.0/24、10.1.1.0/26、10.1.1.1/32被permit，其他路由被deny。

说明：网络号为10.1.1.0，掩码长度在24-32之间的路由会被permit。

✓ 示例3:

```
ip ip-prefix test index 10 permit 10.1.1.0 24 greater-equal 26
```

匹配结果：路由10.1.1.0/26、10.1.1.1/32被permit，其他路由被deny。

说明：网络号为10.1.1.0，掩码长度在26-32之间的路由会被permit。

✓ 示例4:

```
ip ip-prefix test index 10 permit 10.1.1.0 24 greater-equal 26 less-equal 32
```

匹配结果：路由10.1.1.0/26、10.1.1.1/32被permit，其他路由被deny。

说明：网络号为10.1.1.0，掩码长度在26-32之间的路由会被permit。此示例效果与示例3相同。

-----示例5-6为通配地址(0.0.0.0)匹配示例-----

通配地址0.0.0.0表示不限定网络号，只需要匹配掩码范围即可。表2中列出了几种特殊的通配地址。

| 特殊的通配地址                    | 含义         |
|----------------------------|------------|
| 0.0.0.0 0                  | 表示只匹配缺省路由  |
| 0.0.0.0 0 less-equal 32    | 表示匹配所有路由   |
| 0.0.0.0 0 greater-equal 32 | 表示匹配所有主机路由 |

表2 特殊的通配地址

说明：地址前缀列表采用默认拒绝的匹配原则，在创建了一个或多个deny模式的表项后，需要创建一个permit 0.0.0.0 0 less-equal 32表项，允许所有其他路由通过。

## ✓ 示例5:

```
ip ip-prefix test index 10 permit 0.0.0.0 8 less-equal 32
```

匹配结果：5条路由均被permit。

说明：所有掩码长度在8-32之间的路由都被permit

## ✓ 示例6:

```
ip ip-prefix test index 10 deny 10.1.1.0 24
ip ip-prefix test index 20 permit 0.0.0.0 0 less-equal 32
```

匹配结果：只有路由10.1.1.0/24被deny，其他路由都被permit。

说明：路由10.1.1.0/24匹配前缀列表test中索引10节点，但匹配模式是deny，因此结果是deny；

索引20节点permit 0.0.0.0 0 less-equal 32表示允许所有路由通过，因此未匹配上索引节点10的路由都匹配上了索引20节点，均被permit。

叨叨了这么多，能看到这里的都是学霸哟，是不是已经信心满满，掌握了地址前缀列表啦！

地址前缀列表（ip ip-prefix）能过滤出想要的路由，但是要实现了对路由的控制，例如控制路由信息的接收、发布、引入等，还需要在filter-policy或者route-policy中调用地址前缀列表才能实现。下一期我们将介绍如何通过filter-policy实现路由过滤，敬请期待~

## 4 filter-policy 原理与应用

在路由策略专题第一期（初识路由策略）中，我们曾经提到filter-policy也属于路由策略的范畴，但是filter-policy的过滤规则和使用方法与route-policy又有很大的不同，因此这一期我们单独讨论一下filter-policy的原理和应用。

### 4.1 filter-policy 介绍

filter-policy也是一个很常用的路由信息过滤工具，如图1所示，SwitchA、SwitchB、SwitchC之间运行某种路由协议，路由在各个设备之间传递，当需要根据实际需求过滤某些路由信息的时候可以使用filter-policy实现。

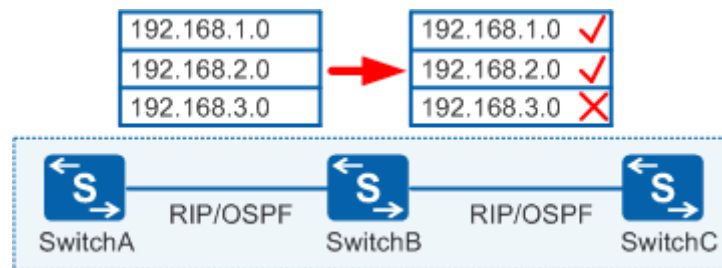


图1 filter-policy的应用场景

**filter-policy使用注意事项：**

- filter-policy是常用于控制路由接收、发布的一个工具。
- filter-policy只能过滤路由信息，无法过滤LSA。
- filter-policy只能过滤路由信息，不能修改路由属性值。

在路由策略专题第一期（初识路由策略）中，我们说过路由策略的主要功能有两个，1）过滤路由信息，2）修改路由属性值。从上面的注意事项来看，其实filter-policy只具备过滤路由信息这个功能。

### 4.2 filter-policy 过滤路由信息的规则

由于距离矢量路由协议（例如RIP）和链路状态路由协议（例如OSPF）原理上的差异，filter-policy应用在这两种路由协议的时候过滤规则也有所不同。如表1所示，在讨论之前我们有必要回忆一下距离矢量路由协议和链路状态路由协议路由信息传递的区别。

| 路由类型 | 路由传递原理 |
|------|--------|
|------|--------|

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| 距离矢量路由协议<br>(例如RIP)  | 各路由设备之间传递的是路由信息。这种路由信息对于报文来说相当于“路标”，设备依靠“路标”来指导报文转发，路标指向哪里报文就转发到哪里。             |
| 链路状态路由协议<br>(例如OSPF) | 各路由设备之间传递的是LSA信息。LSA信息的集合（LSDB）形成整个网络的拓扑结构，相当于一张地图，设备依靠“地图+最短路径算法”为报文找到最佳的转发路径。 |

表1 距离矢量路由协议和链路状态路由协议原理对比

如图2所示，在距离矢量路由协议中，设备之间传递的是路由信息，如果需要对这种路由信息进行某种过滤，可以使用filter-policy实现，出方向和入方向的生效位置如图2所示。如果要过滤掉上游设备到下游设备的路由，只要在上游设备配filter-policy export或者在下游设备上配置filter-policy import就可以了。

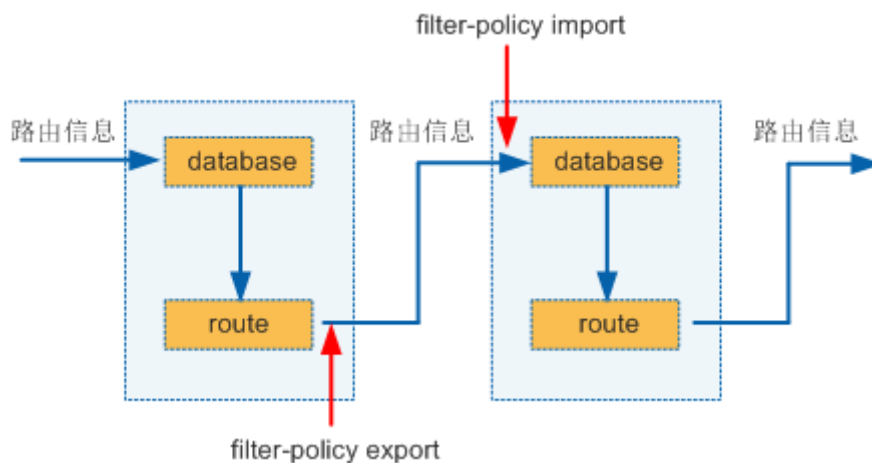


图2 filter-policy在距离矢量路由协议中应用

如图3所示，在链路状态路由协议中，各路由设备之间传递的是LSA信息，然后设备根据LSA汇总成的LSDB信息计算出路由表。

以OSPF为例，filter-policy生效规则如下：

- filter-policy import命令实际上是对OSPF计算出来的路由进行过滤，不是对发布和接收的LSA进行过滤。
- filter-policy export命令用来对引入的路由在发布时进行过滤，只将满足条件的外部路由转换为Type5 LSA（AS-external-LSA）并发布出去。这样可以在引入外部路由时进行特定的过滤，防止形成路由环路。

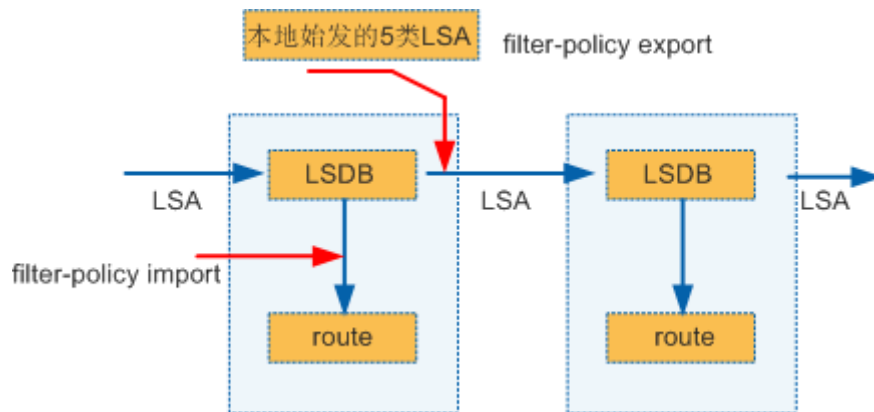


图3 filter-policy在链路状态路由协议中应用

### 4.3 使用 filter-policy 过滤 RIP 路由示例

#### 4.3.1 通过 filter-policy 对 RIP 发布的路由做过滤

##### 需求描述

如图4所示，三台交换机通过RIP交互路由，在SwitchA的RIP进程中引入了三条静态路由（作为测试路由），用户要求在SwitchB上部署filter-policy export，将其通告给SwitchC的路由进行过滤，拒绝192.168.3.0/24这条路由，其他路由放行。

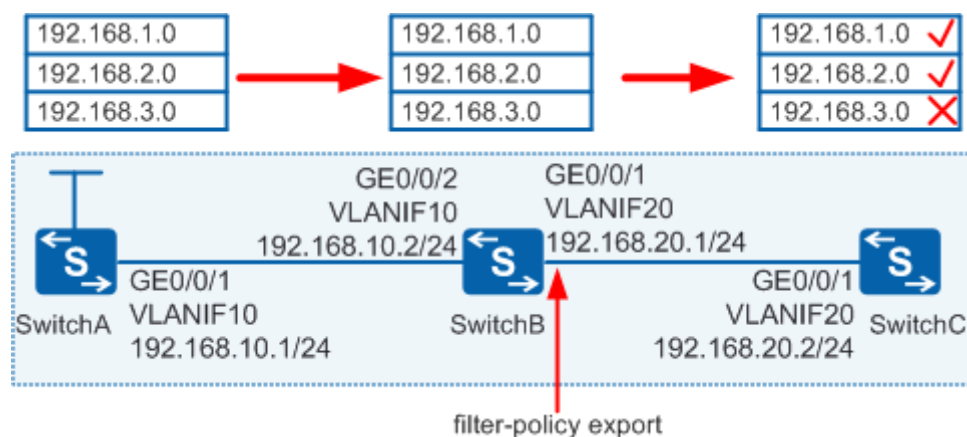


图4 通过filter-policy对RIP发布的路由做过滤

##### 配置方法

1、在SwitchB上定义一个地址前缀列表，“抓取”符合条件的路由。

```
[SwitchB] ip ip-prefix huawei index 10 deny 192.168.3.0 24 //拒绝这条
[SwitchB] ip ip-prefix huawei index 20 permit 0.0.0.0 0 less-equal 32//允许所有
```

2、在SwitchB的RIP视图中，部署filter-policy export。

```
[SwitchB] rip
```



```
[SwitchB-rip-1] filter-policy ip-prefix huawei export Vlanif20
```

## 结果验证

做完上述配置以后，查看SwitchB和SwitchC的路由表如下：

```
[SwitchB] display ip routing-table
```

Route Flags: R - relay, D - download to fib

-----  
Routing Tables: Public

Destinations : 9 Routes : 9

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.1.0/24   | RIP    | 100 | 1    | D     | 192.168.10.1 | Vlanif10    |
| 192.168.2.0/24   | RIP    | 100 | 1    | D     | 192.168.10.1 | Vlanif10    |
| 192.168.3.0/24   | RIP    | 100 | 1    | D     | 192.168.10.1 | Vlanif10    |
| 192.168.10.0/24  | Direct | 0   | 0    | D     | 192.168.10.2 | Vlanif10    |
| 192.168.10.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif10    |
| 192.168.20.0/24  | Direct | 0   | 0    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.20.1/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif20    |

```
[SwitchC] display ip routing-table
```

Route Flags: R - relay, D - download to fib

-----  
Routing Tables: Public

Destinations : 7 Routes : 7

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.1.0/24   | RIP    | 100 | 2    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.2.0/24   | RIP    | 100 | 2    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.10.0/24  | RIP    | 100 | 1    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.20.0/24  | Direct | 0   | 0    | D     | 192.168.20.2 | Vlanif20    |
| 192.168.20.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif20    |

从上面的实验结果来看，在SwitchB上部署filter-policy export以后，SwitchB仍然拥有完整的路由表，而SwitchC已经不能通过RIP学习到192.168.3.0/24这条路由，因为这条路由在SwitchB上发布的时候已经被过滤掉。

### 4.3.2 通过 filter-policy 对 RIP 接收的路由做过滤

#### 需求描述

如图5所示，三台交换机通过RIP交互路由，在SwitchA的RIP进程中引入了三条静态路由（作为测试路由），用户要求在SwitchB上部署filter-policy import，把192.168.3.0/24这条路由拒绝，其他路由放行。

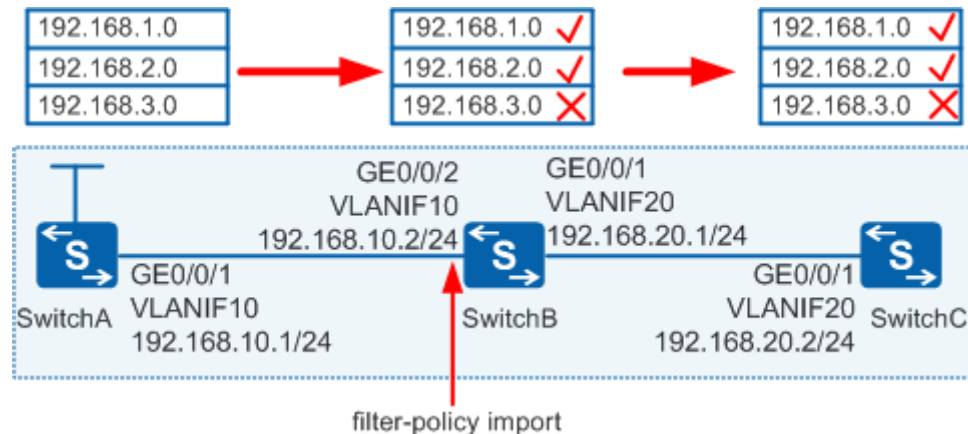


图5 通过filter-policy对RIP接收的路由做过滤

#### 配置方法

1、在SwitchB上定义一个地址前缀列表，“抓取”符合条件的路由。

```
[SwitchB] ip ip-prefix huawei index 10 deny 192.168.3.0 24 //拒绝这条
[SwitchB] ip ip-prefix huawei index 20 permit 0.0.0.0 0 less-equal 32//允许所有
```

2、在SwitchB的RIP视图中，部署filter-policy import。

```
[SwitchB] rip
[SwitchB-rip-1] filter-policy ip-prefix huawei import Vlanif10
```

#### 结果验证

做完上述配置以后，查看SwitchB和SwitchC的路由表如下：

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib

-----
Routing Tables: Public
      Destinations : 8          Routes : 8

Destination/Mask    Proto    Pre  Cost           Flags NextHop         Interface
-----
127.0.0.0/8         Direct   0    0              D    127.0.0.1         InLoopBack0
127.0.0.1/32        Direct   0    0              D    127.0.0.1         InLoopBack0
```

|                 |        |     |   |   |              |          |
|-----------------|--------|-----|---|---|--------------|----------|
| 192.168.1.0/24  | RIP    | 100 | 1 | D | 192.168.10.1 | Vlanif10 |
| 192.168.2.0/24  | RIP    | 100 | 1 | D | 192.168.10.1 | Vlanif10 |
| 192.168.10.0/24 | Direct | 0   | 0 | D | 192.168.10.2 | Vlanif10 |
| 192.168.10.2/32 | Direct | 0   | 0 | D | 127.0.0.1    | Vlanif10 |
| 192.168.20.0/24 | Direct | 0   | 0 | D | 192.168.20.1 | Vlanif20 |
| 192.168.20.1/32 | Direct | 0   | 0 | D | 127.0.0.1    | Vlanif20 |

```
[SwitchC] display ip routing-table
```

Route Flags: R - relay, D - download to fib

---

Routing Tables: Public

Destinations : 7 Routes : 7

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.1.0/24   | RIP    | 100 | 2    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.2.0/24   | RIP    | 100 | 2    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.10.0/24  | RIP    | 100 | 1    | D     | 192.168.20.1 | Vlanif20    |
| 192.168.20.0/24  | Direct | 0   | 0    | D     | 192.168.20.2 | Vlanif20    |
| 192.168.20.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif20    |

从上面的实验结果来看，在SwitchB上部署filter-policy import以后，SwitchB和SwitchC的路由表中都不存在192.168.3.0/24这条路由了。

由于RIP是距离矢量路由协议，它是将自己的路由表通告给它的邻居路由器，当SwitchB的路由表中192.168.3.0/24这条路由被过滤以后，SwitchC也就无法再通过RIP学习到这条路由。也就是说，对于距离矢量路由协议，如果一台设备的路由表被进行了过滤，那么它会继续影响它下游的设备的路由表。

### 注意事项：

filter-policy export和filter-policy import命令在RIP进程下配置，如果基于接口或者协议对路由进行过滤，则一个接口或协议只能配置一个策略；在没有指定接口和协议的情况下，就认为是配置全局过滤策略，同样只能配置一个策略，如果重复配置，新的策略将覆盖之前的策略。

## 4.4 使用 filter-policy 过滤 OSPF 路由示例

### 4.4.1 通过 filter-policy 对 OSPF 接收的路由过滤（区域内）

#### 需求描述

如图6所示，三台交换机同属于OSPF Area 0区域，SwitchA发布测试网段10.1.1.0/24，要求在SwitchB上部署filter-policy import，使得SwitchB的路由表中不允许出现10.1.1.0/24这条路由。

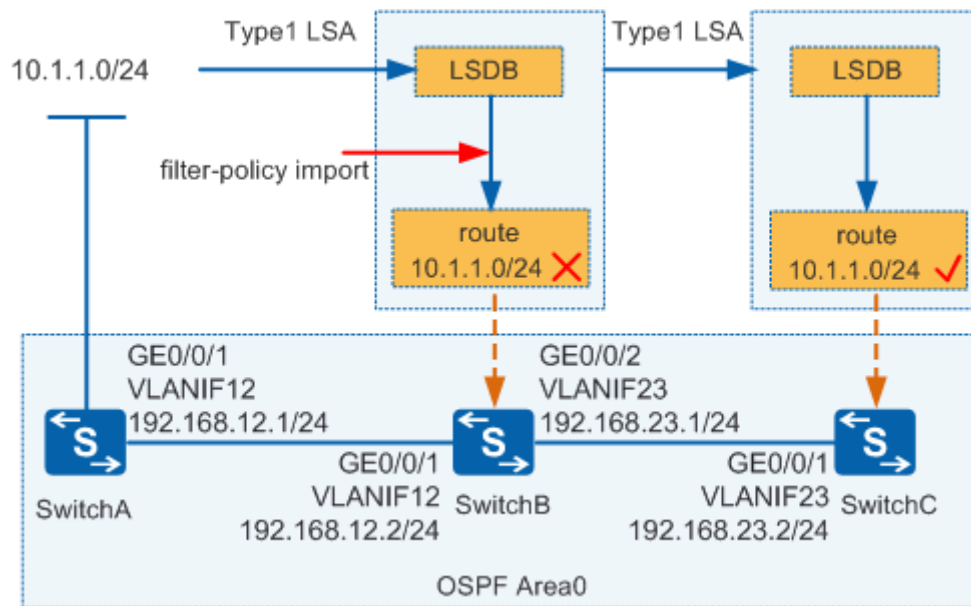


图6 通过filter-policy对OSPF接收的路由过滤（区域内）

#### 配置方法

1、在SwitchB上定义一个地址前缀列表，“抓取”符合条件的路由。

```
[SwitchB] ip ip-prefix huawei index 10 deny 10.1.1.0 24 //拒绝这条
[SwitchB] ip ip-prefix huawei index 20 permit 0.0.0.0 0 less-equal 32//允许所有
```

2、在SwitchB的OSPF视图中，部署filter-policy import。

```
[SwitchB] ospf
[SwitchB-ospf-1] filter-policy ip-prefix huawei import
```

#### 结果验证

做完上述配置以后，查看SwitchB和SwitchC的路由表如下：

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
    Destinations : 6          Routes : 6
```



| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.12.0/24  | Direct | 0   | 0    | D     | 192.168.12.2 | Vlanif12    |
| 192.168.12.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif12    |
| 192.168.23.0/24  | Direct | 0   | 0    | D     | 192.168.23.1 | Vlanif23    |
| 192.168.23.1/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif23    |

```
[SwitchC] display ip routing-table
Route Flags: R - relay, D - download to fib
-----
-
Routing Tables: Public
    Destinations : 6          Routes : 6

Destination/Mask    Proto   Pre  Cost   Flags NextHop         Interface
-----
10.1.1.0/24 OSPF    10   3       D   192.168.23.1   Vlanif23
127.0.0.0/8 Direct  0     0       D   127.0.0.1      InLoopBack0
127.0.0.1/32 Direct  0     0       D   127.0.0.1      InLoopBack0
192.168.12.0/24 OSPF    10   2       D   192.168.23.1   Vlanif23
192.168.23.0/24 Direct  0     0       D   192.168.23.2   Vlanif23
192.168.23.2/32 Direct  0     0       D   127.0.0.1      Vlanif23
```

通过SwitchB和SwitchC的路由表可以看到，虽然在SwitchB上10.1.1.0/24这条路由已经被过滤掉，但是LSA信息会继续传递给SwitchC，所以SwitchC的路由表中继续存在10.1.1.0/24这条路由。这样的结果也验证了我们一开始在注意事项中给出的结论：在链路状态路由协议中，filter-policy只能过滤路由信息，不能过滤LSA信息。

同时SwitchB和SwitchC的LSDB中仍然存在描述10.1.1.0/24这条路由的LSA信息，为了验证这一点，我们看一下SwitchB和SwitchC的LSA信息。

```
[SwitchB] display ospf lsdb router

OSPF Process 1 with Router ID 10.10.10.2
    Area: 0.0.0.0
    Link State Database
.....

Type      : Router
Ls id     : 10.10.10.1
Adv rtr   : 10.10.10.1
```

```

Ls age      : 139
Len         : 48
Options     : E
seq#        : 80000005
chksum      : 0x41c4
Link count: 2
* Link ID: 10.1.1.0
  Data      : 255.255.255.0
  Link Type: StubNet
  Metric    : 1
  Priority  : Low
* Link ID: 192.168.12.2
  Data      : 192.168.12.1
  Link Type: TransNet
  Metric    : 1
```

```
[SwitchC] display ospf lsdb router
```

```

    OSPF Process 1 with Router ID 10.10.10.3
          Area: 0.0.0.0
    Link State Database
.....
```

```

Type       : Router
Ls id      : 10.10.10.1
Adv rtr    : 10.10.10.1
Ls age     : 81
Len        : 48
Options    : E
seq#       : 80000005
chksum     : 0x41c4
Link count: 2
* Link ID: 10.1.1.0
  Data      : 255.255.255.0
  Link Type: StubNet
  Metric    : 1
  Priority  : Low
* Link ID: 192.168.12.2
  Data      : 192.168.12.1
  Link Type: TransNet
  Metric    : 1
```

可以看到SwitchB和SwitchC的LSDB中仍然存在描述10.1.1.0/24这条路由的LSA信息。

#### 4.4.2 通过 filter-policy 对 OSPF 接收的路由过滤（区域间）

如图7所示，相对于上一个场景，这个场景的区别之处是划分了两个不同的区域，SwitchB和SwitchC之间传递的是Type3 LSA，这个Type3 LSA是SwitchB上根据区域间路由生成的。配置方法跟上一个场景一样，不再赘述，我们直接看一下实验结果。

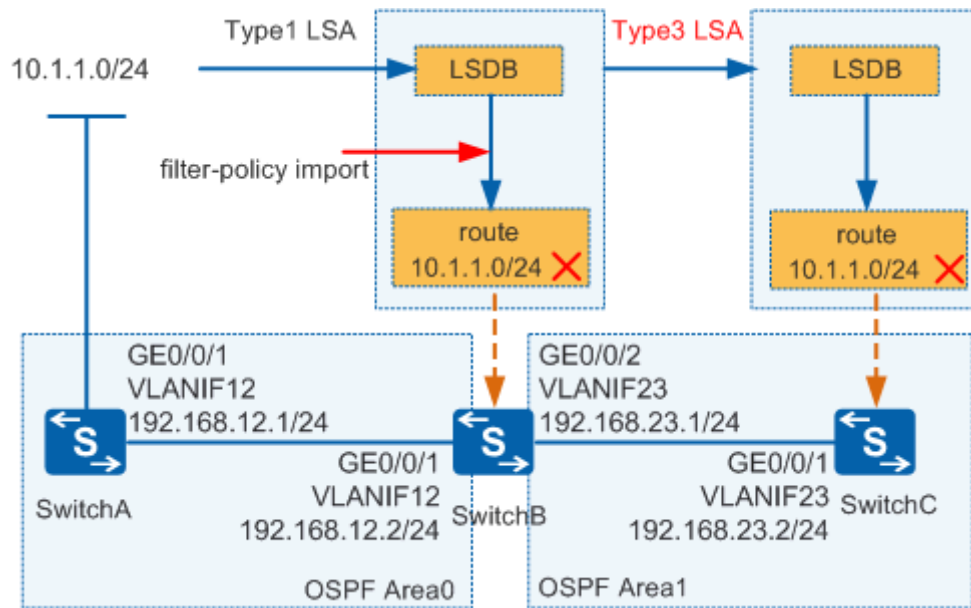


图7 通过filter-policy对OSPF接收的路由过滤（区域间）

在部署过滤策略之前我们先看一下SwitchB和SwitchC的路由表

```
[SwitchB] display ip routing-table
Route Flags: R - relay, D - download to fib

-----
Routing Tables: Public
    Destinations : 7          Routes : 7

Destination/Mask    Proto   Pre  Cost           Flags NextHop         Interface
-----
10.1.1.0/24        OSPF    10   2              D    192.168.12.1      Vlanif12
127.0.0.0/8        Direct  0     0              D    127.0.0.1         InLoopBack0
127.0.0.1/32       Direct  0     0              D    127.0.0.1         InLoopBack0
192.168.12.0/24    Direct  0     0              D    192.168.12.2      Vlanif12
192.168.12.2/32    Direct  0     0              D    127.0.0.1         Vlanif12
192.168.23.0/24    Direct  0     0              D    192.168.23.1      Vlanif23
192.168.23.1/32    Direct  0     0              D    127.0.0.1         Vlanif23

[SwitchC] display ip routing-table
```



Route Flags: R - relay, D - download to fib

Routing Tables: Public

Destinations : 6 Routes : 6

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 10.1.1.0/24      | OSPF   | 10  | 3    | D     | 192.168.23.1 | Vlanif23    |
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.12.0/24  | OSPF   | 10  | 2    | D     | 192.168.23.1 | Vlanif23    |
| 192.168.23.0/24  | Direct | 0   | 0    | D     | 192.168.23.2 | Vlanif23    |
| 192.168.23.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif23    |

可以看到，在部署路由过滤策略之前，SwitchB和SwitchC的路由表中都有10.1.1.0/24这条路由。

## 结果验证

在SwitchB上部署filter-policy import以后，查看SwitchB和SwitchC的路由表，结果如下：

[SwitchB] display ip routing-table

Route Flags: R - relay, D - download to fib

Routing Tables: Public

Destinations : 6 Routes : 6

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop      | Interface   |
|------------------|--------|-----|------|-------|--------------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1    | InLoopBack0 |
| 192.168.12.0/24  | Direct | 0   | 0    | D     | 192.168.12.2 | Vlanif12    |
| 192.168.12.2/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif12    |
| 192.168.23.0/24  | Direct | 0   | 0    | D     | 192.168.23.1 | Vlanif23    |
| 192.168.23.1/32  | Direct | 0   | 0    | D     | 127.0.0.1    | Vlanif23    |

[SwitchC] display ip routing-table

Route Flags: R - relay, D - download to fib

Routing Tables: Public

Destinations : 5 Routes : 5

| Destination/Mask | Proto  | Pre | Cost | Flags | NextHop   | Interface   |
|------------------|--------|-----|------|-------|-----------|-------------|
| 127.0.0.0/8      | Direct | 0   | 0    | D     | 127.0.0.1 | InLoopBack0 |
| 127.0.0.1/32     | Direct | 0   | 0    | D     | 127.0.0.1 | InLoopBack0 |



|                 |        |    |   |   |              |          |
|-----------------|--------|----|---|---|--------------|----------|
| 192.168.12.0/24 | OSPF   | 10 | 2 | D | 192.168.23.1 | Vlanif23 |
| 192.168.23.0/24 | Direct | 0  | 0 | D | 192.168.23.2 | Vlanif23 |
| 192.168.23.2/32 | Direct | 0  | 0 | D | 127.0.0.1    | Vlanif23 |

由于现在划分不同的区域，SwitchC上的10.1.1.0/24这条路由是由SwitchB根据自身学习的路由产生的Type3-LSA描述的，而SwitchB上的这条路由被过滤掉了，因此不能够再产生描述区域间路由的这个Type3-LSA，因此SwitchC上不会再学习到10.1.1.0/24这条路由。

## 5 BGP 路由策略(上)

在前几期的路由策略专题中，我们介绍路由策略的时候，大多是用某种IGP协议进行举例的。实际上，对于BGP路由的控制是路由策略的最大的用武之地，因为我们知道，BGP路由最大的特点就是灵活可控，这主要得益于BGP单独提供了不少工具用于配合执行路由策略。对BGP路由合理使用路由策略，可以做到对路由的精准控制，这也是很多网络工程师追求的一个目标。本期及下期内容内容，我们将会用较大的篇幅讨论BGP路由策略，希望能让大家有所收获。

### 5.1 ip-prefix 在 BGP 中的应用

在BGP中，ip-prefix除了可以配合filter-policy或者route-policy使用以外，还可以被peer命令直接调用，这也是BGP不同于其他路由协议的地方。通过peer命令调用的时候，仅对这一个对等体生效，因此BGP对路由的控制能做到更加精准。我们看下面这个举例。

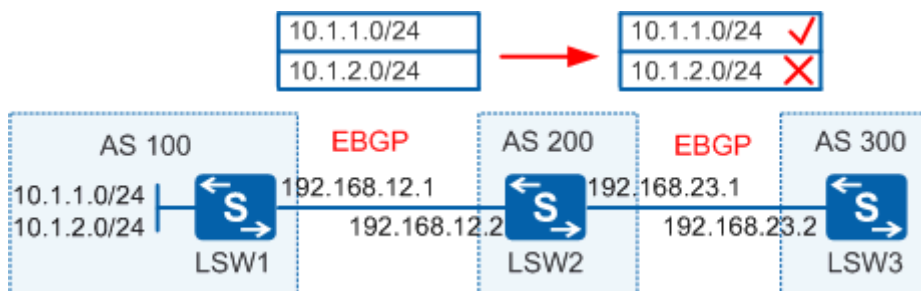


图1 ip-prefix在BGP中的应用

#### 需求描述

如图1所示，LSW1、LSW2、LSW3三个交换机分别属于不同的AS，他们之间维持EBGP邻居关系，在LSW1上通过BGP发布两条路由。现在LSW3上由于某种业务需求，要求禁止10.1.2.0/24这条路由，允许其他的路由。

#### 配置方法

上述需求可以在LSW3的BGP进程中，通过peer命令调用ip-prefix进行过滤。

LSW3上的关键配置如下：

```
#
ip ip-prefix huawei index 10 deny 10.1.2.0 24 //定义 ip-prefix，禁止目标路由
ip ip-prefix huawei index 20 permit 0.0.0.0 0 less-equal 32 //允许其他所有路由
#
bgp 300
 router-id 3.3.3.3
 peer 192.168.23.1 as-number 200
#
ipv4-family unicast
 undo synchronization
 peer 192.168.23.1 enable
 peer 192.168.23.1 ip-prefix huawei import //在 import 方向关联 ip-prefix。
#
```

当然，也可以在LSW2的export方向关联ip-prefix进行路由过滤，效果是一样的，大家可以自行试一下，这里不再赘述。

## 结果验证

完成上述配置后，查看LSW3的BGP路由表，可以看到，只有10.1.1.0/24这条路由，10.1.2.0/24已经被过滤掉。

```
[LSW3] display bgp routing-table

BGP Local router ID is 3.3.3.3
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 1

   Network                NextHop         MED           LocPrf        PrefVal Path/Ogn
*>  10.1.1.0/24            192.168.23.1              0           200 100i
```

## 5.2 filter-policy 在 BGP 中的应用

filter-policy这个过滤工具我们前面曾经介绍过，我们知道filter-policy应用在距离矢量路由协议和链路状态路由协议的时候，过滤规则是不一样的，BGP属于一种距离矢量路由协议，具体过滤规则请参见路由策略专题第4期。

filter-policy在BGP中有两种应用方式：

- 在BGP视图下对全局应用，对所有对等体生效。
- 通过peer命令应用，仅对这一个对等体生效。

相对于其他路由协议，BGP可以针对某个对等体单独应用filter-policy，这样也保证了BGP对路由的控制更加灵活、精确。下面我们来看一个具体的举例。

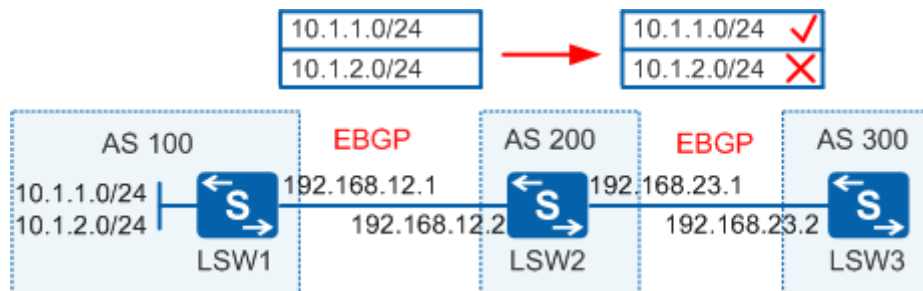


图2 filter-policy在BGP中的应用

### 需求描述

如图2所示，LSW1、LSW2、LSW3三个交换机分别属于不同的AS，他们之间维持EBGP邻居关系，在LSW1上通过BGP发布两条路由。现在LSW3上由于某种业务需求，要求禁止10.1.2.0/24这条路由，允许其他的路由。

### 配置方法

- 方法一：在BGP视图下对全局应用，对所有对等体生效。

LSW3的关键配置如下：

```
#
acl number 2001          //定义 ACL，用于匹配目标路由
rule 5 deny source 10.1.2.0 0    //拒绝 10.1.2.0 这条路由
rule 10 permit source any        //允许所有
#
bgp 300
router-id 3.3.3.3
peer 192.168.23.1 as-number 200
#
ipv4-family unicast
undo synchronization
filter-policy 2001 import //对所有对等体使用 filter-policy 过滤路由
peer 192.168.23.1 enable
#
```

- 方法二：通过peer命令应用，仅对这一个对等体生效。

LSW3的关键配置如下：

```
#
acl number 2001          //定义 ACL，用于匹配目标路由
rule 5 deny source 10.1.2.0 0      //拒绝 10.1.2.0 这条路由
rule 10 permit source any        //允许所有
#
bgp 300
router-id 3.3.3.3
peer 192.168.23.1 as-number 200
#
ipv4-family unicast
undo synchronization
peer 192.168.23.1 enable
peer 192.168.23.1 filter-policy 2001 import //对单个对等体使用 filter-policy
#
```

### 5.3 route-policy 在 BGP 中的应用

route-policy在前面我们也详细介绍过了，在BGP中，route-policy同样是一个非常重要的工具，可以用来设置路由的属性，对路由进行过滤等。

我们再通过一个举例来看一下在BGP中调用route-policy的方法。

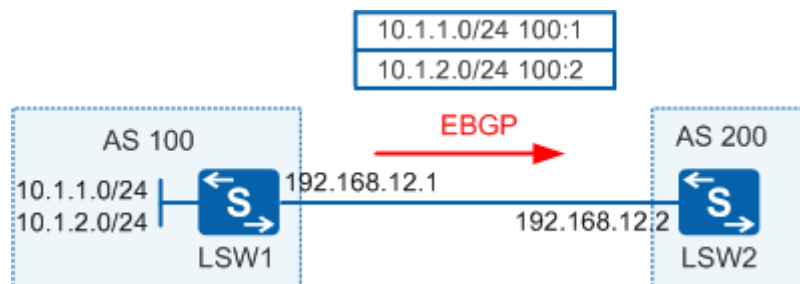


图3 route-policy在BGP中的应用

#### 需求描述

如图2所示，LSW1和LSW2两个交换机分别属于不同的AS，他们之间维持EBGP邻居关系，在LSW1上通过BGP发布两条路由。现在由于某种业务需求，需要对LSW1发布的两条路由分别设置不通的团体属性值，10.1.1.0/24这条路由设置团体属性值为100:1，10.1.2.0/24这条路由设置团体属性值为100:2。

#### 配置方法

- 方法一：通过network命令调用route-policy。

这种配置方法是在通过network命令发布路由的时候直接调用route-policy，设置相应的团体属性值以后再发布给对等体。

LSW1的关键配置如下：

```
#
ip ip-prefix 1 index 10 permit 10.1.1.0 24 //定义前缀列表，匹配目标路由
ip ip-prefix 2 index 10 permit 10.1.2.0 24
#
route-policy huawei permit node 10 //为不同的路由设置不同的团体属性值
  if-match ip-prefix 1
  apply community 100:1
#
route-policy huawei permit node 20
  if-match ip-prefix 2
  apply community 100:2
#
route-policy huawei permit node 30 //允许剩余所有路由
#
bgp 100
  router-id 1.1.1.1
  peer 192.168.12.2 as-number 200
#
  ipv4-family unicast
    undo synchronization
    network 10.1.1.0 255.255.255.0 route-policy huawei
    network 10.1.2.0 255.255.255.0 route-policy huawei
    peer 192.168.12.2 enable
    peer 192.168.12.2 advertise-community
#
```

● 方法二：通过peer命令调用route-policy。

这种配置方法，在通过network命令发布路由的时候不使用route-policy，但是通过peer命令针对对等体使用route-policy，这样路由在发布之前先通过route-policy的过滤，通过并设置相应的团体属性值以后再发布给对等体。

LSW1的关键配置如下：

```
#
ip ip-prefix 1 index 10 permit 10.1.1.0 24 //定义前缀列表，匹配目标路由
ip ip-prefix 2 index 10 permit 10.1.2.0 24
#
route-policy huawei permit node 10 //为不同的路由设置不同的团体属性值
  if-match ip-prefix 1
```

```
apply community 100:1
#
route-policy huawei permit node 20
  if-match ip-prefix 2
  apply community 100:2
#
route-policy huawei permit node 30    //允许剩余所有路由
#
bgp 100
  router-id 1.1.1.1
  peer 192.168.12.2 as-number 200
  #
  ipv4-family unicast
    undo synchronization
    network 10.1.1.0 255.255.255.0
    network 10.1.2.0 255.255.255.0
    peer 192.168.12.2 enable
    peer 192.168.12.2 route-policy huawei export
    peer 192.168.12.2 advertise-community
  #
```

## 结果验证

上述两种配置方法虽然不一样，但是对于这个场景，都可以实现10.1.1.0/24这条路由设置团体属性值为100:1，10.1.2.0/24这条路由设置团体属性值为100:2。完成配置后，在LSW2上查看一下BGP路由表，可以看到已实现上述需求。

```
[LSW2] display bgp routing-table community

BGP Local router ID is 2.2.2.2
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
               Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 2

   Network          NextHop      MED      LocPrf  PrefVal Community
-----
*>  10.1.1.0/24      192.168.12.1    0                0      <100:1>
*>  10.1.2.0/24      192.168.12.1    0                0      <100:2>
```

相对于其他路由协议，BGP中应用route-policy的命令会更多一点，更具体一点。常见的调用route-policy的命令有下面这些：

- network

- peer
- import-route
- dampening

上面给大家举例看了一下network和peer命令调用route-policy的方法，其他的调用命令我们不再一一举例。

对于BGP路由，更重要的是通过BGP的路由属性对路由进行控制，BGP具有丰富的路由属性可供使用，常用的有AS\_Path过滤器和团体属性过滤器，这个我们将会在下一期路由策略专题中详细介绍，请大家持续关注。

## 6 BGP 路由策略(下)

前面我们介绍的路由策略基本都是基于路由前缀的，但是我们知道BGP的“用武之地”主要是一些大型骨干网络，在这种网络中一般路由前缀的规模会非常大，如果继续基于路由前缀执行路由策略，那么工作量将会大的不可想象。能否跳出路由前缀的“怪圈”，换一种角度去“抓取”感兴趣的路由呢？对于BGP路由协议来说是可以实现的，本期我们主要介绍一下AS\_Path过滤器和团体属性过滤器在BGP路由策略中的使用方法。

### 6.1 AS\_Path\_Filter 在 BGP 中的应用

#### 6.1.1 AS\_Path\_Filter 及正则表达式

##### AS\_Path过滤器（AS\_Path\_Filter）

AS\_Path属性按矢量顺序记录了某条路由从本地到目的地址所要经过的所有AS编号。如图1所示，某条BGP路由的AS\_Path属性实际上可以看作是一个包含空格的字符串，所以可以通过正则表达式来进行匹配。



图1 BGP路由的AS\_Path属性

正则表达式就是用一个“字符串”来描述一个特征，然后去验证另一个“字符串”是否符合这个特征。BGP的AS\_Path过滤器主要是定义AS\_Path正则表达式，然后去匹配BGP路由的AS\_Path属性信息，从而实现对BGP路由信息的过滤。

例如ip as-path-filter 1 permit 495就定义了一个AS\_Path过滤器1，使用的正则表达式是495，这个表达式的含义是匹配任何包含495的字符串。

这么来看，AS\_Path过滤器的核心内容就是正则表达式。关于正则表达式的内容较为复杂，我们这里仅讨论一些跟AS\_Path过滤器相关的内容。

##### AS\_Path正则表达式的组成

AS\_Path过滤器使用正则表达式来定义匹配规则。正则表达式由元字符和数值两部分组成：



- 元字符定义了匹配的规则。
- 数值定义了匹配的对象。

BGP AS\_Path正则表达式支持的元字符如表1所示。

| 元字符    | 含义                                              | 实例                                                                                                                                                                                                              |
|--------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .      | 匹配除“\n”之外任何单个字符，包括空格。                           | .*表示匹配任意字符串，即 AS_Path 为任意，可以用来匹配所有路由。通常定义了多个 deny 模式的 ip as-path-filter 子句之后，会定义一个 ip as-path-filter as-path-filter-name permit .*子句，用于允许其他路由通过。                                                                |
| *      | 之前的字符在目标对象中出现 0 次或连续多次。                         | 参考上例。                                                                                                                                                                                                           |
| +      | 之前的字符在目标对象中出现 1 次或连续多次。                         | 65+表示 6 在 AS_Path 的首位，而 5 在 AS_Path 中出现一次或多次，那么： <ul style="list-style-type: none"> <li>● 如下字符串都符合这个特征：65，655，6559，65259，65529 等。</li> <li>● 如下字符串不符合这个特征：56，556，5669，55269，56259 等。</li> </ul>                 |
|        | 竖线左边和右边的字符为“或”的关系。                              | 100 65002 65003 表示匹配 100、65002 或 65003。                                                                                                                                                                         |
| ^      | 之后的字符串必须出现在目标对象的开始。                             | ^65 表示匹配以 65 开头的字符串，那么： <ul style="list-style-type: none"> <li>● 如下字符串都符合这个特征：65，651，6501，65001 等。</li> <li>● 如下字符串不符合这个特征：165，1650，6650，60065 等。</li> </ul>                                                    |
| \$     | 之前的字符串必须出现在目标对象的结束。                             | 65\$表示匹配以 65 结尾的字符串，那么： <ul style="list-style-type: none"> <li>● 如下字符串都符合这个特征：65，165，1065，10065，60065 等。</li> <li>● 如下字符串不符合这个特征：651，1650，6650，60650，65001 等。</li> </ul> ^\$表示匹配空字符串，即 AS_Path 为空，通常用来匹配本地始发路由。 |
| (xyz)  | 一对圆括号内的正则表达式作为一个子正则表达式，匹配子表达式并获取这一匹配。圆括号内也可以为空。 | 100(200)+可以匹配 100200、100200200、.....                                                                                                                                                                            |
| [xyz]  | 匹配方括号内列出的任意字符。                                  | [896]表示匹配含有 8、9 或 6 中任意一个字符。                                                                                                                                                                                    |
| [^xyz] | 匹配除了方括号内列出的字符外的任意字符(^号在字符前)。                    | [^896]表示匹配含有 8、9 或 6 这几个字符之外的任意一个字符。                                                                                                                                                                            |

|        |                                                                  |                                                                                                                                                                                                                                                                                                                                   |
|--------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [a-z]  | 匹配指定范围内的任意字符。                                                    | [2-4]表示匹配 2, 3, 4; [0-9]表示匹配数字 0~9。                                                                                                                                                                                                                                                                                               |
| [^a-z] | 匹配不在指定范围内的任意字符。                                                  | [^2-4]表示匹配除 2, 3, 4 外的其他字符; [^0-9]表示匹配除数字 0~9 外的其他字符。                                                                                                                                                                                                                                                                             |
| -      | 匹配一个符号,包括逗号、左大括号、右大括号、左括号、右括号和空格,在表达式的开头或结尾时还可作起始符、结束符(同 ^, \$)。 | <ul style="list-style-type: none"> <li>● ^65001_表示匹配字符串的开始为 65001,字符串的后面为符号,也即 AS_Path 最左边 AS(最后一个 AS)为 65001,可以用来匹配 AS 65001 邻居发送的路由,</li> <li>● _65001_表示匹配字符串里有 65001,即 AS_Path 中有 65001,可以用来匹配经过 AS 65001 的路由。</li> <li>● _65001\$表示匹配字符串的最后为 65001,字符串前面是符号,即 AS_Path 最右边 AS(起始 AS)为 65001,可以用来匹配 AS 65001 始发的路由。</li> </ul> |
| \      | 转义字符。                                                            | 用来将紧跟其后的元字符转变为普通字符                                                                                                                                                                                                                                                                                                                |

表1 AS\_Path正则表达式支持的元字符

### 6.1.2 AS\_Path\_Filter 的应用方式

AS\_Path过滤器只定义一个过滤工具,需要在某个地方调用这个过滤工具才会最终生效。在 BGP中可以有两种方式调用AS\_Path过滤器: 1、通过peer命令直接调用as-path-filter, 2、通过route-policy调用AS\_Path\_Filter。

#### 应用方式一: 通过peer命令直接调用as-path-filter

```
#
ip as-path-filter s1 permit ^100$
#
bgp 65100
 peer 10.1.1.2 as-path-filter s1 import
#
```

在应用方式1中,我们定义了一个AS\_Path\_Filter并关联了一个正则表达式^100\$,这个AS\_Path\_Filter可以匹配AS\_PATH严格为100的路由(不包含任何其他AS号或其他数字),随后我们将AS\_Path\_Filter应用在了peer命令上,这样一来,只有被AS\_Path\_Filter 1所匹配的路由,才会被传递给BGP邻居10.1.1.2。

#### 应用方式二: 通过route-policy调用as-path-filter

```
#
ip as-path-filter s1 permit ^100$
#
route-policy huawei permit node 10
```

```

if-match as-path-filter s1
apply local-preference 100
#
bgp 65100
peer 10.1.1.2 route-policy huawei import
#

```

在应用方式2中，我们在route-policy中的if-match命令调用定义好的AS\_Path\_Filter，随后使用apply命令设置LP路径属性值，并在BGP配置模式下应用在了peer命令上（import方向）。这样，该交换机在收到BGP邻居10.1.1.2所发送过来的BGP路由中，所有被AS\_Path\_Filter匹配的路由均将LP路径属性值设置为100。

### 6.1.3 AS\_Path\_Filter 的使用举例

上面介绍完了AS\_Path\_Filter的匹配规则和应用方式，接下来我们通过一个示例来具体看一下在BGP中如何使用AS\_Path\_Filter进行路由过滤。

如图2所示，LSW1与LSW2之间，LSW1与LSW3之间，LSW2与LSW3之间，LSW2与LSW4之间，LSW3与LSW4之间，LSW4与LSW5之间都建立EBGP邻居。各个设备把LoopBack0接口的IP地址通过network命令发布到BGP中，用作测试路由网段。

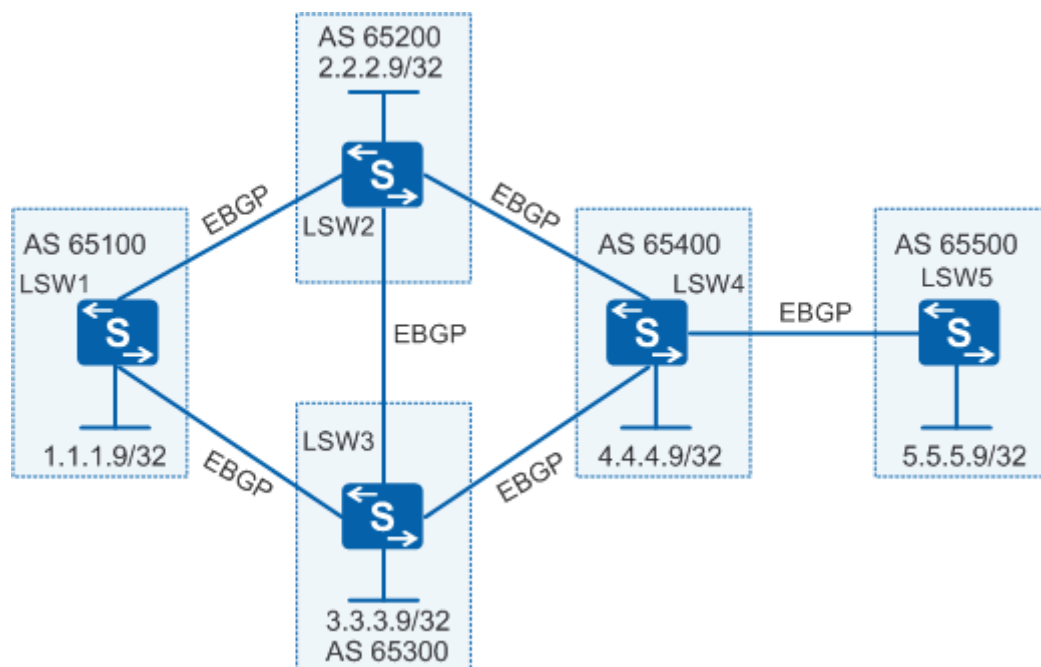


图2 使用AS\_Path\_Filter对BGP路由进行过滤

当没有使用AS\_Path过滤器时，LSW1的原始BGP路由表如下。

```

[LSW1] display bgp routing-table

```

```
BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
              h - history, i - internal, s - suppressed, S - Stale
              Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 9
```

|    | Network    | NextHop  | MED | LocPrf | PrefVal | Path/Ogn           |
|----|------------|----------|-----|--------|---------|--------------------|
| *> | 1.1.1.9/32 | 0.0.0.0  | 0   |        | 0       | i                  |
| *> | 2.2.2.9/32 | 10.1.1.2 | 0   |        | 0       | 65200i             |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65200i       |
| *> | 3.3.3.9/32 | 10.1.2.2 | 0   |        | 0       | 65300i             |
| *  |            | 10.1.1.2 |     |        | 0       | 65200 65300i       |
| *> | 4.4.4.9/32 | 10.1.1.2 |     |        | 0       | 65200 65400i       |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65400i       |
| *> | 5.5.5.9/32 | 10.1.1.2 |     |        | 0       | 65200 65400 65500i |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65400 65500i |

**Case1:** 定义一个AS\_Path过滤器s1，只接收AS 65500始发的路由。

```
[LSW1] ip as-path-filter s1 permit _65500$ //定义一个 AS_Path 过滤器 s1
[LSW1] bgp 65100
[LSW1-bgp] ipv4-family unicast
[LSW1-bgp-af-ipv4] peer 10.1.1.2 as-path-filter s1 import //通过 peer 命令调用
[LSW1-bgp-af-ipv4] peer 10.1.2.2 as-path-filter s1 import
```

配置完成后BGP路由表如下：

```
[LSW1] display bgp routing-table

BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
              h - history, i - internal, s - suppressed, S - Stale
              Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 3
```

|    | Network    | NextHop  | MED | LocPrf | PrefVal | Path/Ogn           |
|----|------------|----------|-----|--------|---------|--------------------|
| *> | 1.1.1.9/32 | 0.0.0.0  | 0   |        | 0       | i                  |
| *> | 5.5.5.9/32 | 10.1.1.2 |     |        | 0       | 65200 65400 65500i |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65400 65500i |

从以上显示信息可以看出，AS 65500始发的路由被允许，其他路由被拒绝。

**Case2:** 定义一个AS\_Path过滤器s2, 拒绝AS 65500始发的路由, 允许接收其他路由。

```
[LSW1] ip as-path-filter s2 deny _65500$
[LSW1] ip as-path-filter s2 permit .*
[LSW1] bgp 65100
[LSW1-bgp] ipv4-family unicast
[LSW1-bgp-af-ipv4] peer 10.1.1.2 as-path-filter s2 import
[LSW1-bgp-af-ipv4] peer 10.1.2.2 as-path-filter s2 impor
```

配置完成后BGP路由表如下:

```
[LSW1]display bgp routing-table

BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 7
```

|    | Network    | NextHop  | MED | LocPrf | PrefVal | Path/Ogn     |
|----|------------|----------|-----|--------|---------|--------------|
| *> | 1.1.1.9/32 | 0.0.0.0  | 0   |        | 0       | i            |
| *> | 2.2.2.9/32 | 10.1.1.2 | 0   |        | 0       | 65200i       |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65200i |
| *> | 3.3.3.9/32 | 10.1.2.2 | 0   |        | 0       | 65300i       |
| *  |            | 10.1.1.2 |     |        | 0       | 65200 65300i |
| *> | 4.4.4.9/32 | 10.1.1.2 |     |        | 0       | 65200 65400i |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65400i |

从以上显示信息可以看出, AS 65500始发的路由被拒绝, 其他路由被允许。

**Case3:** 定义一个AS\_Path过滤器s3, 拒绝经过AS 65400的路由

```
[LSW1] ip as-path-filter s3 deny _65400_
[LSW1] ip as-path-filter s3 permit .*
[LSW1] bgp 65100
[LSW1-bgp] ipv4-family unicast
[LSW1-bgp-af-ipv4] peer 10.1.1.2 as-path-filter s3 import
[LSW1-bgp-af-ipv4] peer 10.1.2.2 as-path-filter s3 import
```

配置完成后BGP路由表如下:

```
[LSW1] display bgp routing-table

BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete
```

Total Number of Routes: 5

|    | Network    | NextHop  | MED | LocPrf | PrefVal | Path/Ogn     |
|----|------------|----------|-----|--------|---------|--------------|
| *> | 1.1.1.9/32 | 0.0.0.0  | 0   |        | 0       | i            |
| *> | 2.2.2.9/32 | 10.1.1.2 | 0   |        | 0       | 65200i       |
| *  |            | 10.1.2.2 |     |        | 0       | 65300 65200i |
| *> | 3.3.3.9/32 | 10.1.2.2 | 0   |        | 0       | 65300i       |
| *  |            | 10.1.1.2 |     |        | 0       | 65200 65300i |

从以上显示信息可以看出，经过AS 65400的路由被拒绝，其他路由被允许。

**Case4:** 定义一个AS\_Path过滤器s4，拒绝经过AS 65400的路由，其中AS 65400既不是路由的始发AS，也不是路由经过的最后一个AS。

```
[LSW1] ip as-path-filter s4 deny ._65400_.
[LSW1] ip as-path-filter s4 permit .*
[LSW1] bgp 65100
[LSW1-bgp] ipv4-family unicast
[LSW1-bgp-af-ipv4] peer 10.1.1.2 as-path-filter s4 import
[LSW1-bgp-af-ipv4] peer 10.1.2.2 as-path-filter s4 import
```

配置完成后BGP路由表如下：

```
[LSW1] display bgp routing-table

BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 7

Network          NextHop          MED    LocPrf    PrefVal    Path/Ogn
*> 1.1.1.9/32     0.0.0.0          0              0          i
*> 2.2.2.9/32     10.1.1.2         0              0          65200i
*                10.1.2.2         0              0          65300 65200i
*> 3.3.3.9/32     10.1.2.2         0              0          65300i
*                10.1.1.2         0              0          65200 65300i
*> 4.4.4.9/32     10.1.1.2         0              0          65200 65400i
*                10.1.2.2         0              0          65300 65400i
```

从以上显示信息可以看出，只有AS\_Path的中间AS是65400的路由被拒绝，其他路由被允许。

**Case5:** 定义一个AS\_Path过滤器s5，匹配本地始发路由，不接收其他AS的任何路由。

```
[LSW1] ip as-path-filter s5 permit ^$
[LSW1] bgp 65100
[LSW1-bgp] ipv4-family unicast
[LSW1-bgp-af-ipv4] peer 10.1.1.2 as-path-filter s5 import
[LSW1-bgp-af-ipv4] peer 10.1.2.2 as-path-filter s5 import
```

配置完成后BGP路由表如下：

```
[LSW1] display bgp routing-table

BGP Local router ID is 10.1.1.1
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
               Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 1

   Network          NextHop         MED      LocPrf    PrefVal Path/Ogn
---
*>  1.1.1.9/32      0.0.0.0          0                0          i
```

从以上显示信息可以看出，只有AS\_Path为空的本地始发路由被允许，其他路由被拒绝。

## 6.2 Community 属性在 BGP 中的应用

### 6.2.1 Community 属性介绍

#### Community属性的作用

在讨论Community属性之前，我们先说一点题外话。相信大家在外出旅游的时候有时候会发现，某个旅行团的游客通常会统一带一个“**小红帽**”，上面写着某某团队，这样导游就很方便的对这个团队进行引导和管理，比如要上车了，导游只需要喊一嗓子：“某某团队的，上车了”，而不需要逐个喊每个游客的名字。这就相当于为这一类游客打上一个相同的标签（团体属性），后续的引导和管理就统一用这个标签进行，大大提高了效率和安全性。

其实在BGP路由里面，也有一个类似“**小红帽**”的属性。就是Community属性，即团体属性。

- Community属性是一组4个字节的数值，RFC1997规定前两个字节表示AS号，后两个字节表示基于管理目的设置的标示符，格式为AA: NN。
- Community属性是一种BGP路由标记，用于简化路由策略的执行。可以将某些路由分配一个特定的Community属性值，之后就可以基于Community值而不是每条路由来抓取路由并执行相应的策略了。

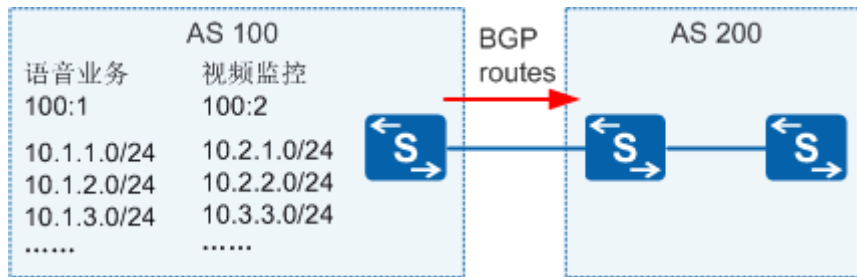


图3 Community属性的应用场景

在图3中，AS100内有大量的路由被引入BGP，这些路由分别用于语音通话和视频监控两种业务。路由通过BGP通告给AS200。现在AS200的设备基于某种需求，需要分别对语音通话和视频监控的路由执行不同的策略，那么怎么匹配这些路由呢？可以用ACL或者ip-prefix逐条匹配路由，但是由于路由前缀非常多，这样工作量太大，效率低下。

可以使用Community属性来解决这个问题。在AS100引入这些路由的时候，就分别打上相应的Community值用来区分语音通话的路由和视频监控的路由，凡是语音通话的路由，就打上标记100:1，凡是视频监控的路由就打上标记100:2，那么这些属性值随着路由传递给了AS200，在AS200上需要分别对语音通话和视频监控的路由做策略的时候，只需要抓取相应Community值即可。例如抓取100:1的community值也就抓取了所有语音业务的路由。

### BGP公认的团体属性

BGP定义了一些公认的团体属性，这些团体属性可以直接使用，常见的如表2所示：

| 团体属性名称              | 说明                                                                |
|---------------------|-------------------------------------------------------------------|
| internet            | 缺省情况下，所有的路由都属于 Internet 团体。具有此属性的路由可以被通告给所有的 BGP 对等体。             |
| no-advertise        | 具有此属性的路由在收到后，不能被通告给任何其他的 BGP 对等体。                                 |
| no-export           | 具有此属性的路由在收到后，不能被发布到本地 AS 之外。如果使用了联盟，则不能被发布到联盟之外，但可以发布给联盟中的其他子 AS。 |
| no-export-subconfed | 具有此属性的路由在收到后，不能被发布到本地 AS 之外，也不能发布到联盟中的其他子 AS。                     |

表2 BGP公认团体属性

### 6.2.2 设置路由的 Community 属性

要使用团体属性过滤器，前提是路由携带了Community属性。就好比游客头上戴的“小红帽”，如果没有这个“小红帽”，那么所有基于这个“小红帽”的策略都无法执行。所以我们先来



看一下如何设置路由的Community属性，我们通过下面这个举例来看一下。

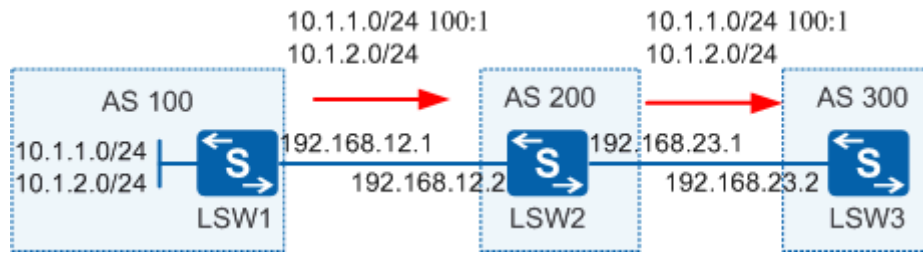


图4 设置路由前缀的Community属性

### 需求描述

如图4所示，在AS100内的LSW1上通过BGP发布两条路由：10.1.1.0/24和10.1.2.0/24。在LSW1的BGP进程中，使用出方向（export）路由策略，修改10.1.1.0/24这条路由的Community属性为100:1，这样下游设备就可以根据这个Community属性执行相应的策略。

### 配置方法

R1上的关键配置：

```
#
ip ip-prefix huawei index 10 permit 10.1.1.0 24 //定义一个前缀列表，匹配目标路由
#
route-policy RP permit node 10 //定义一个路由策略，设置目标路由的团体属性
  if-match ip-prefix huawei
  apply community 100:1
route-policy RP permit node 20 //用于允许剩余所有路由通过
#
bgp 100
  router-id 1.1.1.1
  peer 192.168.12.2 as-number 200
#
ipv4-family unicast
  undo synchronization
  network 10.1.1.0 255.255.255.0 //发布路由
  network 10.1.2.0 255.255.255.0 //发布路由
  peer 192.168.12.2 enable
  peer 192.168.12.2 route-policy RP export //对 BGP 对等体的 export 方向绑定路由策略
  peer 192.168.12.2 advertise-community //配置将团体属性发布给对等体
#
```

R2上的关键配置：

```
#
bgp 200
```

```
router-id 2.2.2.2
peer 192.168.12.1 as-number 100
peer 192.168.23.2 as-number 300
#
ipv4-family unicast
undo synchronization
peer 192.168.12.1 enable
peer 192.168.23.2 enable
peer 192.168.23.2 advertise-community //配置将团体属性发布给对等体
#
```

值得注意的是，默认情况下，Community属性是不会随路由前缀更新给BGP peer的，因此在LSW1和LSW2上都需要通过peer advertise-community配置将团体属性发布给对等体功能。

### 结果验证

完成上述配置之后，我们在LSW3上查看一下BGP路由，结果如下：

```
<LSW3> display bgp routing-table 10.1.1.0

BGP local router ID : 3.3.3.3
Local AS number : 300
Paths: 1 available, 1 best, 1 select
BGP routing table entry information of 10.1.1.0/24:
From: 192.168.23.1 (2.2.2.2)
Route Duration: 01h08m07s
Direct Out-interface: Vlanif23
Original nexthop: 192.168.23.1
Qos information : 0x0
Community:<100:1> //这个团体属性就是在 Lsw1 上通过路由策略设置，并传递到下游的
AS-path 200 100, origin igp, pref-val 0, valid, external, best, select, active,
pre 255
Not advertised to any peer yet
```

从LSW3的BGP路由表项可以看到，在LSW1上设置路由的Community属性以后，经过路由传递，到达LSW3上的时候携带一个Community属性，这样，在LSW3上就可以根据这个标记执行某些策略了。

关于设置BGP路由的Community属性，还有更详细的设置方法，这里我们就不再一一列举，举几个常见的例子，供大家参考，如表3所示：

| 命令                      | 功能                               |
|-------------------------|----------------------------------|
| apply community 100     | 团体名更改为 100。                      |
| apply community 100 150 | 团体名更改为 100 或 150，即 BGP 路由属于两个团体。 |

|                                  |                                           |
|----------------------------------|-------------------------------------------|
| apply community 100 150 additive | 在原来基础上追加 100 和 150 两个团体属性。即 BGP 路由属于三个团体。 |
| apply community none             | 删除 BGP 路由的团体属性。                           |

表3 设置BGP路由的Community属性举例

### 6.2.3 使用 Community-filter 匹配 BGP 路由

上一小节，我们介绍了如何设置BGP路由的Community属性，当上游的路由器设置了Community属性以后，这个属性会随着路由传递下来，这样下游的设备就可以根据这个属性执行相应的策略。BGP提供了一个工具：community-filter，即团体属性过滤器。下面我们通过一个例子来看一下如何使用团体属性过滤器。

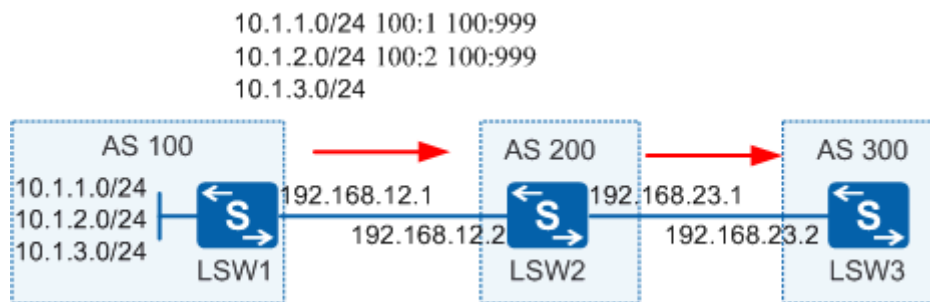


图5 使用community-filter匹配BGP路由

#### 需求描述

如图5所示，AS100内的LSW1发布三条BGP路由，Community属性的设置如图所示，这个Community属性随着路由的传递到达AS300内的LSW3设备上。执行策略之前我们先看一下LSW3的BGP路由表：

```
<LSW3> display bgp routing-table
```

```
BGP Local router ID is 3.3.3.3
Status codes: * - valid, > - best, d - damped,
              h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete
```

```
Total Number of Routes: 3
```

|    | Network     | NextHop      | MED | LocPrf | PrefVal | Path/Ogn |
|----|-------------|--------------|-----|--------|---------|----------|
| *> | 10.1.1.0/24 | 192.168.23.1 |     |        | 0       | 200 100i |
| *> | 10.1.2.0/24 | 192.168.23.1 |     |        | 0       | 200 100i |

```
*> 10.1.3.0/24      192.168.23.1      0      200 100i
```

可以看到，LSW3上的BGP路由表里有三条路由。现在LSW3上根据团体属性执行路由策略，允许携带100:999属性的路由通过，其他的路由不允许通过。

### 配置方法

设置Community属性的方法上面已经讲过，不再赘述，这里只看一下如何使用community-filter过滤路由。

LSW3上的关键配置：

```
#
ip community-filter 1 permit 100:999 //允许包含团体属性 100:999 的路由
#
route-policy huawei permit node 10 //通过路由策略调用团体属性过滤器
  if-match community-filter 1
#
bgp 300
  router-id 3.3.3.3
  peer 192.168.23.1 as-number 200
#
  ipv4-family unicast
    undo synchronization
    peer 192.168.23.1 enable
    peer 192.168.23.1 route-policy huawei import //对 peer 使用入方向路由策略
#
```

### 结果验证

完成上述配置以后，查看LSW3的BGP路由表：

```
[LSW3] display bgp routing-table

BGP Local router ID is 3.3.3.3
Status codes: * - valid, > - best, d - damped,
               h - history, i - internal, s - suppressed, S - Stale
Origin : i - IGP, e - EGP, ? - incomplete

Total Number of Routes: 2
   Network          NextHop      MED       LocPrf    PrefVal Path/Ogn
*> 10.1.1.0/24      192.168.23.1      0         200 100i
*> 10.1.2.0/24      192.168.23.1      0         200 100i
```

由于定义的community-filter匹配包含团体属性为100:999的路由，因此10.1.1.0/24，10.1.2.0/24

这两条路由将在LSW3上允许，其他BGP路由被禁止通过。

针对这个场景，我们再给出如下几个常用的community-filter的配置方法，供大家参考：

**Case1:**

```
ip community-filter 1 permit 100:1 100:999
```

需要 100:1 100:999 这两个团体属性都有才能匹配。因此匹配 10.1.1.0/24 这条路由。

**Case2:**

```
ip community-filter 1 permit 100:1
```

```
ip community-filter 1 permit 100:2
```

只要 community 属性中包含 100:1 或者 100:2 就能匹配到。因此匹配 10.1.1.0/24 和 10.1.2.0/24 这两条路由。

**Case3:**

```
ip community-filter 1 permit internet
```

默认所有路由携带 internet 属性，所以这样会匹配所有的路由。

至此，路由策略专题全部结束了，不知道大家是否有所收获。相信每一位数通网络工程师都曾经追求过一个目标：对路由表做到精准控制，这样才能保证业务流量按照规划的流量模型转发。我们希望通过这个技术贴，让大家更加熟练的掌握路由策略的使用方法，控制路由能够更加得心应手。谢谢大家持续关注，欢迎大家积极留言讨论。



# 交换机在江湖

版权所有 © 华为技术有限公司2016 。保留一切权利。